Branch edits in continuous thought

Question are alternatives readable from a latent state also separately usable?

System 2-layer Coconut-style transformer, trained from scratch

Seeds 3 training seeds + pause and no-latent controls **Date** 23 Sep 2026 · follow-up 24 Sep 2026

A continuous-thought transformer can hold several graph-search branches in one latent state, and edits to one branch move that branch more than the others. The edits also pass a semantic test: an inserted branch continues with the recipient's graph rather than carrying the donor's answers. But each edit weakens as more branches are active. The follow-up study (Part II) explains why. Scaling the edit does not help, and nor does restoring attention gain. Two things change instead. First, branches share a normalised, relative code: boosting one lowers the others, and the per-branch signal falls roughly as $W^{-0.6\ldots0.9}$. With it, the model's own sensitivity to a branch's deep nodes collapses as width grows. Second, nodes two or three hops away move from the latent pathway to queries reading the graph tokens directly, where a thought edit cannot reach. Averaged over the branch, the latent pathway's share falls from 76% to 31%. For the deepest nodes, though, a thought edit reproduces a similar share of the model's own counterfactual at every width. The directions a linear probe reads from are almost causally inert; the activation patterns for the same node ids carry the effect.

Part III looks at what the latent loop computes and how that depends on architecture and seed. In the delayed format every recurrent model runs a clocked breadth-first sweep: at step k its first layer reads the edges of one depth across all reachable branches, and blocking those edges at that step, and only then, cuts the branch's deeper nodes. The per-edge attention on the frontier halves each time the width doubles while the clock stays fixed, which is Part II's shared budget seen directly. Seeds differ in clock rate. Those that finish the trained depth inside the budget stop there, and the stop is keyed to the step embedding: holding it at an early step lets two of them follow chains well past the trained depth (up to 88% at seven hops). The seed whose sweep is still running at the last step follows chains beyond any trained length when given more steps, and so do models trained with only four steps, too few for the sweep to finish (both competent seeds), and a model trained with no step embedding at all; a model trained with eight steps does not. A wider and a deeper variant both kept more of each branch in the latent pathway, contrary to both pre-registered predictions for them. With the target given first, the raw-feedback variant instead walks backward from the target along its parent chain, a single-path search whose accuracy does not depend on width. Of the pre-registered Part III tests, 2 passed, 3 failed, and the rest are pending. See the Part III scorecard.

Status. Two completed studies on one architecture and one synthetic task. Part I was frozen from discovery data and confirmed on held-out families. Part II followed a second pre-registration (`docs/PROTOCOL2.md`, `docs/protocol2_frozen.json`) and was confirmed on *fresh* graphs that exclude every family any earlier analysis touched. It uses 300 graphs per width, a bootstrap over graphs within each seed, and three recurrent seeds shown separately. Controls: 3 pause-token and 3 no-latent seeds. Training used a documented lookup scaffold (see Limits). Part III (`docs/PROTOCOL3.md`, addenda B–G) adds three architecture variants, two held-out seeds, test-time search tests and causal attention-blocking tests, several of them exploratory and marked as such.

What the study found

Recurrence buys reliability on the deepest queries, not a broad capability. With depth-matched decoys (chance 50%), recurrent seeds score 59%–66% at six hops. All 6 pause and no-latent control seeds score 50%–56%. Below that, controls vary enormously between seeds. Some never pass two hops. One no-latent seed, with no latent steps at all, reaches 88% at four hops and 67% at five, matching the weakest recurrent seed. Both of Part I's single-seed control conclusions were seed luck.

Readable ≠ usable. Editing along the probe's own read-out directions does nothing, even at a matched perturbation size where the branch's activation patterns move the answer 6–57× more. Probe filters hold 0.2–0.3% of the donor difference; patterns hold 35–53%.

Branch edits are real but weaken with width, and dose does not rescue them. Raising the edit to 1.5× adds little at any width; at eight branches no seed reaches its two-branch effect (P2). The fraction of the clean counterfactual effect a thought edit recovers falls with width in every seed.

Why the branch average weakens. Per deep node the edit does not weaken: at 5–6 hops a thought edit reproduces 45%–98% of the model's own counterfactual at W = 2 and 63%–109% at W = 8. The average falls for two reasons. The model's own counterfactual on deep nodes collapses with width (at four hops 0.86 → 0.26). And two- and three-hop nodes move to query-time graph reading, so the latent-pathway share of the whole branch falls 76% → 31% while query reading rises 37% → 71%. The two routes together reproduce the whole clean effect. Pause-token models split it differently by seed: query-time graph reading carries 45%–97% across pause seeds and widths, with the rest in START's cache.

Edits have the right semantics. An inserted branch continues with the recipient's graph (follow index up to +0.53, CI above zero at every frozen site and width), not the donor's endpoints. Wrong-branch donors do nothing. Two branches edited together each follow their own recipient continuation, slightly sub-additively.

Branches share a normalised, relative code. Boosting one reachable branch's attention lowers the others; boosting a decoy or every edge barely matters; restoring total gain changes nothing. Per-branch amplitude falls as $W^{-\beta}$ with $\beta = 0.62–0.87$, between the $1/\sqrt{W}$ expected from a LayerNorm-normalised superposition and $1/W$.

Scorecard. Of eleven pre-registered predictions, 6 held for every model tested (P1, P2, P3, P7, P9, P14). The rest held only partially or were falsified in part. Notably, local geometry *does* add some held-out predictive value for targeted effects (P8), one seed prefers $1/W$ over $1/\sqrt{W}$ (P10), and a two-hop-limited pause seed keeps branches in START's cache (P1b). See the scorecard.

Part III: one seed learned a search that can run longer. Given more latent steps than it was trained with, seed 2 keeps resolving deeper nodes and follows chains to distances never seen in training (distance 7: 71% after twelve steps). Seeds 0 and 1, the pause and no-latent controls, and the raw-feedback variant stay at chance beyond six hops. Width caps the accuracy the search reaches, not how fast it proceeds.

Part III: the thought carries the frontier; the step embedding carries the stop. Holding the step embedding at step 2 leaves the depth each step reads unchanged but removes the halt, and models that never extrapolate follow chains past six hops (seed 3 and V2 at 87–88% at distance 7). The propagation step generalises; the halt was learned from the step index. The prospective test of this on V1 (P17) failed, so it is not universal.

Part III: the route shift is a property of the delayed diagnostic. With the target given first, the answer token never reads the graph (0% query-time share in all three seeds), and the latent pathway carries the whole effect at every width.

Part III: removing the thought's renormalisation changes the amplitude law as predicted. With raw-residual feedback $\beta = 0.97$ (reference 0.62–0.87), and competition between branches persists (P13, pre-registered).

Setup

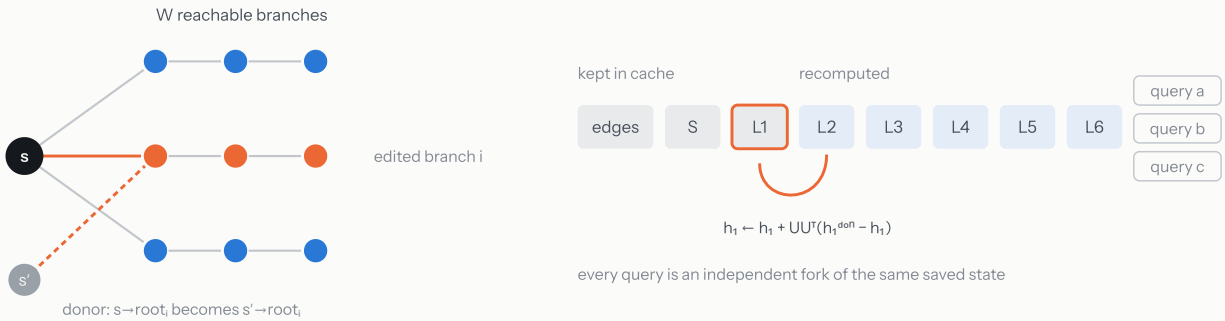
Task. Directed reachability on fixed-budget graphs: 10 disjoint branches, each a random six-node tree, 62 nodes and 60 edge tokens per graph in shuffled order, with ids drawn fresh from 80 per graph. $W \in \{1...8\}$ branches hang off the start node; the rest hang off a decoy hub, so every unreachable node has a shape-matched reachable counterpart. Structural families are hashed into train / discovery / confirm splits so a renamed or reordered copy can never cross splits.

Model. Two-layer pre-LN decoder, $d = 256$, trained from scratch. Coconut-style recurrence: the final-layer output at each of $K = 6$ latent positions becomes the next position's input. In the *delayed* format the graph and start are processed, six latent steps run, and only then is a target node supplied; many targets are asked as independent forks of the identical saved state, which is what makes "several branches in one state" testable.

Intervention. For one reachable branch, the donor is the same graph with that branch's single hub edge moved to the decoy hub. At thought h_k we set $h \leftarrow h + UU^T(h_{donor} - h)$ and recompute everything after position k, keeping the cache before it. U is the full space, a branch-specific pattern subspace (six directions indexed by the edited branch's node ids), the probe's own read-out filters for those ids, a shared PCA subspace of discovery donor differences, or matched random controls. The frozen site k^* is chosen per seed on discovery graphs by a pre-set rule (seed 0: $k^* = 1$; seeds 1–2: $k^* = 0, 0$).

Task and intervention interface

Every graph has 10 six-node branches, 62 nodes and 60 edge tokens; width W only changes which hub a branch hangs from.



Left: W branches hang off the start hub *s*; the rest hang off a decoy hub *s′* and are shape-matched unreachable negatives. The donor differs from the recipient by one edge token (*s*→root$_i$ becomes *s′*→root$_i$). Right: the thought h_k is replaced by $h + UU^T(h_{donor} - h)$ before it becomes the input of latent position k+1; the cache for positions ≤ k is kept, everything after is recomputed, and every node is then queried as an independent fork of that one saved state.

Competence and latent use

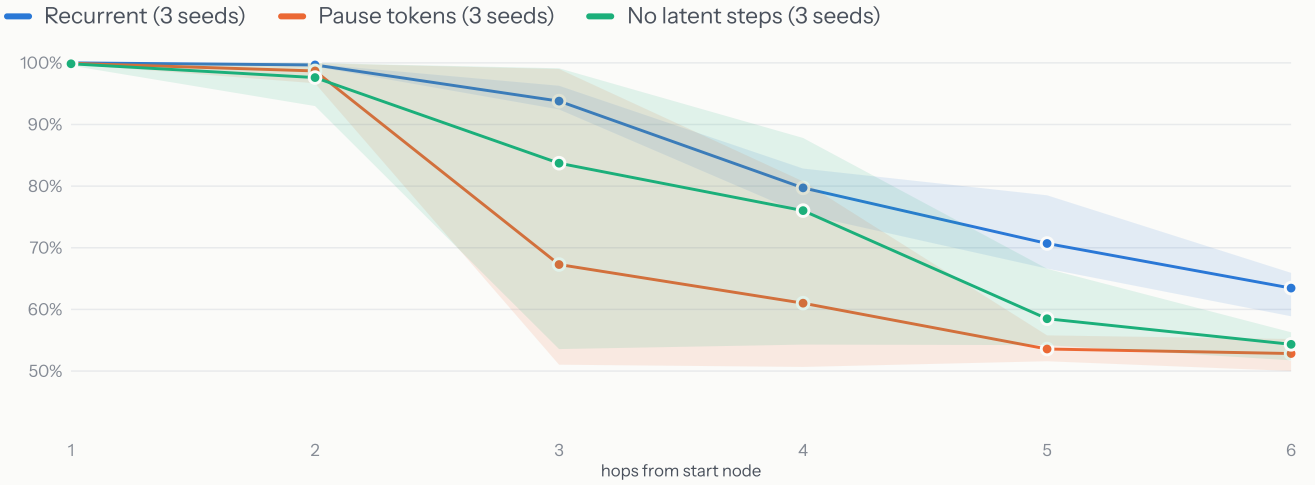
Every recurrent seed clears the 85% competence bar on held-out families (90%–93% overall balanced accuracy). With depth-matched decoys as negatives, they score 67%–79% at five hops and 59%–66% at six. At six hops every control seed is at or near chance (pause 50%–55%, no-latent 52%–56%).

Below six hops the controls differ enormously between seeds. Two of three pause seeds and one no-latent seed never pass two hops. Others solve 3–4 hops through computation in the prefix, and one no-latent seed reaches 67% at five hops, as good as the weakest recurrent seed, with no latent steps at all. Part I's single pause seed happened to be a strong one and its single no-latent seed a weak one, so both control conclusions there were seed luck. What replicates is narrower: recurrent training reached the deepest queries in every seed, and no control seed reached six hops.

Truncating the latent budget at inference is not a clean ablation. All recurrent seeds answer one-hop queries from the last latent position, so dropping any step collapses those queries. That is why the matched controls carry the latent-use claim.

Depth-matched accuracy by distance from the start node

At each distance: balanced accuracy of reachable-branch nodes vs decoy-branch nodes at the same within-branch depth (chance = 50%). Delayed format, held-out families, full latent budget.



Mean over seeds with the seed range as a band. Controls replay the matched recurrent seed's realised curriculum schedule, with the same data and scaffold. Per-distance recall on reachable nodes alone (the Part I metric) can be inflated by a bias toward answering "reachable"; this matched metric cannot.

model	latent budget at inference	delayed bal. acc [95% CI]	upfront bal. acc
no-latent seed 0	0	70% [70%, 70%]	67%
no-latent seed 1	0	91% [91%, 92%]	71%
no-latent seed 2	0	93% [93%, 94%]	86%
pause seed 0	0	75% [74%, 75%]	76%
pause seed 0	1	88% [87%, 88%]	81%
pause seed 0	2	90% [90%, 91%]	81%
pause seed 0	3	91% [91%, 91%]	81%
pause seed 0	4	91% [91%, 91%]	81%
pause seed 0	5	91% [91%, 91%]	82%

model	latent budget at inference	delayed bal. acc [95% CI]	upfront bal. acc
pause seed 0	6	91% [91%, 91%]	81%
pause seed 1	0	58% [57%, 58%]	62%
pause seed 1	1	66% [66%, 67%]	68%
pause seed 1	2	69% [69%, 69%]	68%
pause seed 1	3	71% [70%, 71%]	68%
pause seed 1	4	71% [71%, 72%]	68%
pause seed 1	5	71% [71%, 72%]	68%
pause seed 1	6	71% [71%, 72%]	68%
pause seed 2	0	56% [55%, 56%]	68%
pause seed 2	1	60% [59%, 60%]	71%
pause seed 2	2	65% [64%, 65%]	71%
pause seed 2	3	71% [70%, 71%]	71%
pause seed 2	4	71% [71%, 71%]	71%
pause seed 2	5	72% [71%, 72%]	71%
pause seed 2	6	72% [72%, 72%]	71%
recurrent seed 0	0	74% [74%, 74%]	51%
recurrent seed 0	1	76% [76%, 76%]	76%
recurrent seed 0	2	78% [77%, 78%]	85%
recurrent seed 0	3	79% [79%, 80%]	85%
recurrent seed 0	4	81% [80%, 81%]	85%
recurrent seed 0	5	81% [81%, 82%]	85%
recurrent seed 0	6	90% [90%, 90%]	85%
recurrent seed 1	0	86% [86%, 86%]	50%
recurrent seed 1	1	89% [89%, 89%]	87%
recurrent seed 1	2	90% [90%, 90%]	87%
recurrent seed 1	3	91% [91%, 91%]	87%
recurrent seed 1	4	92% [91%, 92%]	87%
recurrent seed 1	5	92% [91%, 92%]	87%
recurrent seed 1	6	92% [92%, 92%]	87%
recurrent seed 2	0	72% [72%, 73%]	50%
recurrent seed 2	1	79% [79%, 80%]	72%
recurrent seed 2	2	83% [83%, 84%]	73%
recurrent seed 2	3	87% [87%, 88%]	84%
recurrent seed 2	4	90% [90%, 90%]	84%
recurrent seed 2	5	92% [91%, 92%]	88%
recurrent seed 2	6	93% [92%, 93%]	90%

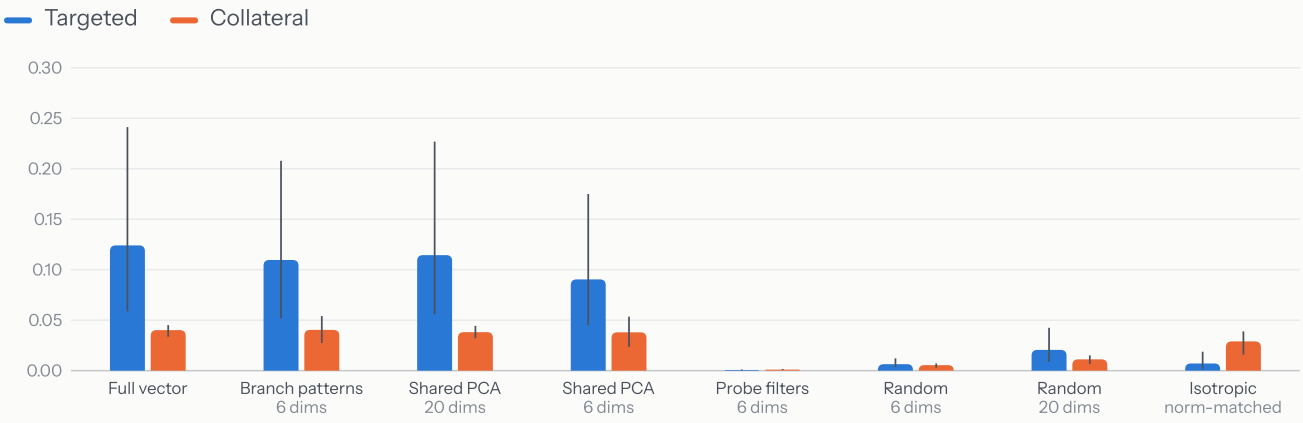
Readable directions are not the usable ones

A linear probe reads each node's reachability from the thought above chance, and above label-shuffled and untrained-model controls, which sit at 0.49–0.52 AUC. So the branches are *readable*. The obvious next step, editing a branch along the probe's read-out directions, fails completely. The probe's weights are *filters* that separate the classes against structured noise. They are nearly orthogonal to the *pattern* along which the thought actually moves when the branch is detached, which is the familiar filter/pattern distinction from neuroimaging decoders (Haufe et al., 2014). Reading the same six node ids through the activation covariance gives branch-specific pattern directions, and those carry most of the full-vector effect.

The effect is also not generic perturbation. A random direction with the full difference's norm moves the targeted branch by at most +0.019 while still producing collateral. Rank-matched random subspaces capture about 2% of the difference and do nothing. A shared 20-dimensional PCA subspace of discovery differences reproduces the full effect.

Single-branch removal at the frozen site, W = 4

Targeted effect: drop in P(reachable) on the edited branch's nodes. Collateral: mean |ΔP| on the other reachable branches. Bars: mean over seeds; whiskers: seed range.



Each seed at its own frozen site k*. "Branch patterns" and "probe filters" use the same six node ids and the same rank; they differ only in whether the covariance is applied. Isotropic = random direction with the norm of the full donor difference.

condition	seed 0: targeted / collateral	seed 1: targeted / collateral	seed
Full vector	+0.059 / 0.034	+0.072 / 0.042	
Branch patterns 6 dims	+0.052 / 0.027	+0.069 / 0.040	
Shared PCA 20 dims	+0.056 / 0.032	+0.060 / 0.038	
Shared PCA 6 dims	+0.045 / 0.024	+0.051 / 0.037	
Probe filters 6 dims	+0.000 / 0.001	+0.001 / 0.002	

condition	seed 0: targeted / collateral	seed 1: targeted / collateral	seed
Random 6 dims	+0.004 / 0.003	+0.004 / 0.006	
Random 20 dims	+0.009 / 0.007	+0.010 / 0.012	
Isotropic norm-matched	+0.002 / 0.016	+0.001 / 0.032	

What width does to readability and use

Across widths, the targeted effect of a single-branch edit falls monotonically from W = 2 to W = 8 in every seed. Collateral on the untouched branches falls too, but not as fast, so the targeted share of the change shrinks. Per-branch probe AUC also declines with width, from 0.58–0.62 at W = 2 to about 0.55 at W = 8. Indexed to W = 2, readability keeps 50% of its above-chance margin at W = 8, while the targeted effect keeps 29%. This is the pattern the proposal set out to test: decodability outlasts demonstrated separate use.

Branch-effect matrices from one saved state

Rows: which reachable branch is edited. Columns: which branch is queried. Diagonal = targeted drop in P(reachable); off-diagonal = |ΔP| on an untouched branch. Full-vector removal at k*, mean over seeds.

W = 2

	q1	q2
edit 1	0.24	0.06
edit 2	0.06	0.23

W = 4

	q1	q2	q3	q4
edit 1	0.12	0.04	0.04	0.04
edit 2	0.04	0.13	0.04	0.04
edit 3	0.04	0.04	0.13	0.04
edit 4	0.04	0.04	0.04	0.12

W = 8

	q1	q2	q3	q4	q5	q6	q7	q8
edit 1	.07	.03	.03	.03	.03	.03	.03	.03
edit 2	.03	.07	.03	.03	.03	.03	.03	.03
edit 3	.03	.03	.07	.03	.03	.03	.03	.03
edit 4	.03	.03	.03	.06	.03	.03	.03	.03
edit 5	.03	.03	.03	.03	.06	.03	.03	.03
edit 6	.03	.03	.03	.03	.03	.07	.03	.03
edit 7	.03	.03	.03	.03	.03	.03	.06	.03
edit 8	.03	.03	.03	.03	.03	.03	.03	.07

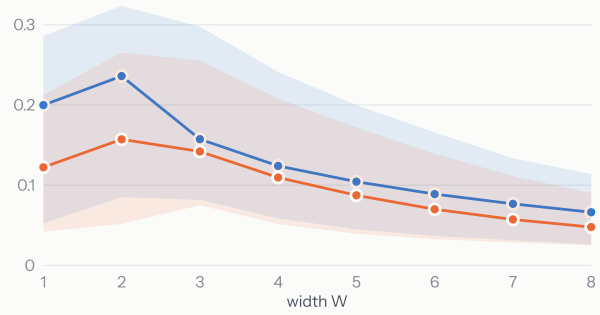
Branch slots are in arbitrary order, so only the diagonal/off-diagonal contrast is meaningful. Mean diagonal 0.236 / 0.124 / 0.066 vs mean off-diagonal 0.059 / 0.040 / 0.032 at W = 2 / 4 / 8.

Effects of a single-branch edit as width grows

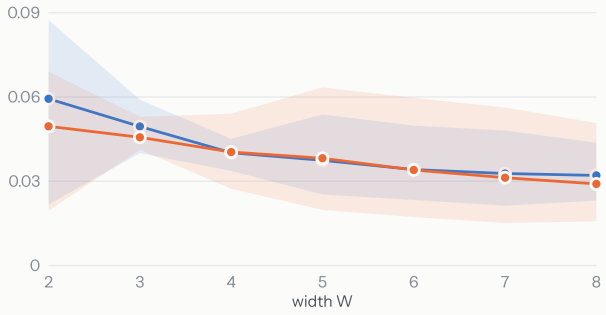
Full-vector and pattern-subspace removal at the frozen site. Lines: mean over seeds; bands: seed range.

Full vector Branch pattern subspace

Targeted effect (ΔP toward counterfactual)



Collateral on other reachable branches (|ΔP|)

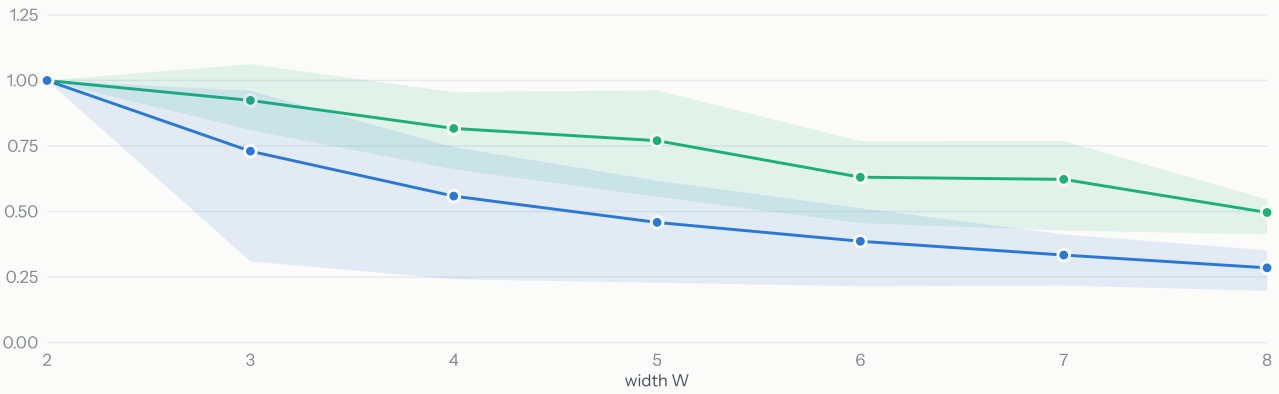


Collateral is undefined at W = 1 (no other reachable branch). Seed ranges are wide: seed 2's effects are 3–4× seed 0's at every width, but all three decline.

Decodability outlasts causal control

Both indexed to W = 1 = 1.0 on one axis: probe per-branch AUC above chance (AUC − 0.5), and the full-vector targeted effect.

Readable: probe AUC − 0.5 Usable: targeted effect



Indexed at W = 2 because W = 1 has no competing branch (and one seed's W = 1 effect is anomalously small). Per-seed index, then mean across seeds; band = seed range.

W	seed 0: full tgt / col · pattern tgt	seed 1: full tgt / col · pattern
1	+0.261 / – · +0.212	+0.286 / – · +0
2	+0.085 / 0.087 · +0.052	+0.300 / 0.069 · +0
3	+0.082 / 0.049 · +0.075	+0.093 / 0.059 · +0
4	+0.059 / 0.034 · +0.052	+0.072 / 0.042 · +0
5	+0.045 / 0.025 · +0.039	+0.068 / 0.033 · +0
6	+0.037 / 0.023 · +0.033	+0.064 / 0.029 · +0
7	+0.032 / 0.021 · +0.029	+0.065 / 0.029 · +0
8	+0.026 / 0.023 · +0.025	+0.059 / 0.029 · +0

Which nodes an edit reaches, and when

Averaging over a whole branch hides most of the effect. At the frozen site, nodes one hop from the start are untouched and nodes two hops away nearly so (≤ 0.05), because they are answered from positions the edit leaves in the cache. The effect sits on nodes three or more hops away, the ones the remaining latent steps must still reach. Timing agrees: edits at the earliest thoughts matter most, and edits at k ≥ 4 do almost nothing in two of three seeds. That fits a frontier that is expanded over the first few steps and then read, and it gives a precise answer to "which alternatives are in play at step k".

Targeted effect by node distance and width

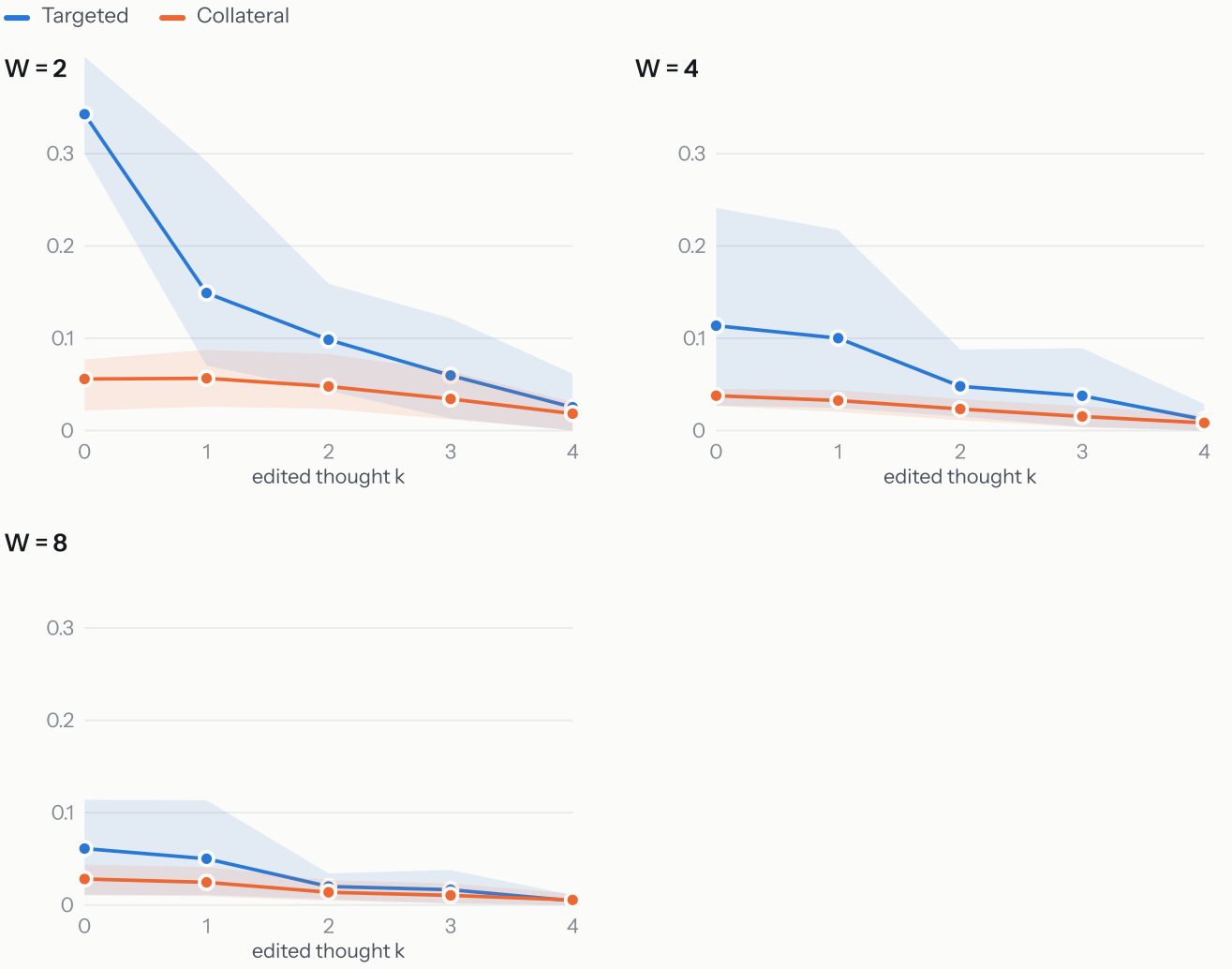
Full-vector removal at the frozen site; mean over seeds. Distance = hops from the start node.

	W = 1	W = 2	W = 3	W = 4	W = 5	W = 6	W = 7	W = 8
1 hop	0.00	0.00	0.00	-0.00	0.00	0.00	0.00	-0.00
2 hops	0.00	0.02	0.00	0.00	0.00	0.00	0.00	0.00
3 hops	0.09	0.33	0.23	0.20	0.18	0.17	0.15	0.13
4 hops	0.47	0.47	0.32	0.25	0.20	0.17	0.14	0.12
5 hops	0.63	0.54	0.38	0.25	0.19	0.14	0.11	0.09
6 hops	0.74	0.38	0.21	0.13	0.11	0.08	0.05	0.05

Mean over seeds, each at its own k*. Distance 1–2 cells are ≈ 0 in every seed. Darker = larger drop in P(reachable) toward the counterfactual "unreachable".

Edit timing: which latent step carries the branch

Full-vector removal applied at thought k = 0…4 (k = 0 is the START output). Mean over seeds.



k = 0 is the START output, before any latent position. Mean over seeds; band = seed range.

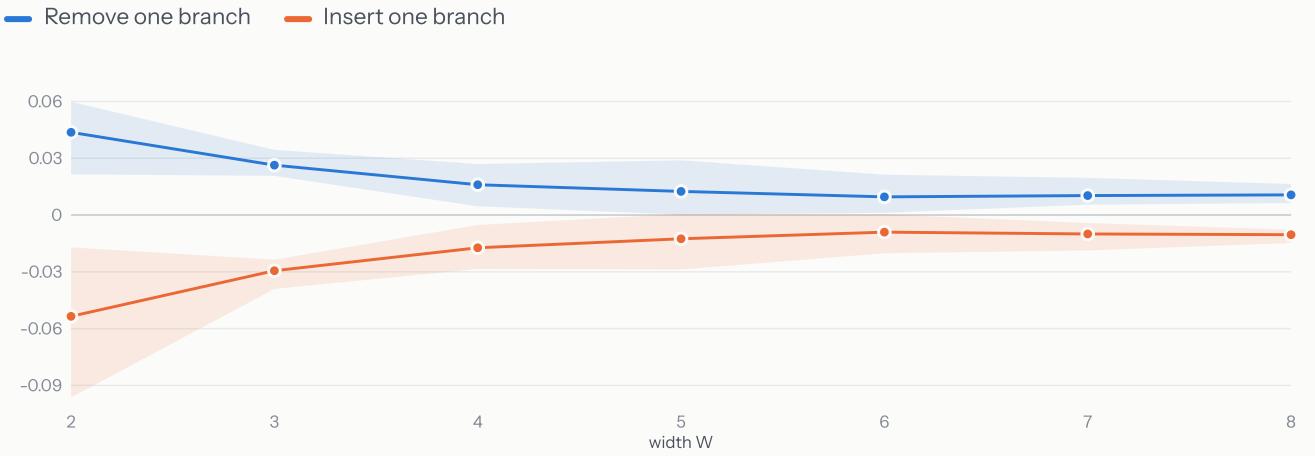
Branches compete for a shared budget

If branches shared the thought as independent slots, removing one should leave the others alone. Instead the others move in a consistent direction. After a removal they become more reachable; after an insertion, less. The signed shift is largest at small width and shrinks roughly like the change in a 1/W share would. This sign pattern was written down as a prediction before any confirmation graph was drawn, and it holds in two seeds at every width and in the third at small and large widths. The identity control is exactly zero and a same-status control (one decoy edge moved, nothing's reachability changed) stays at 0.022 or less, so this is not generic damage.

A direct look at attention agrees (exploratory, discovery graphs). The first layer's attention from the latent positions onto each reachable branch's edges, above the decoy baseline, is split among the branches. Each branch's share falls at least as fast as 1/W. A fixed attention budget divided among the alternatives is a simple mechanism for the interference.

Signed collateral: what happens to the *other* branches

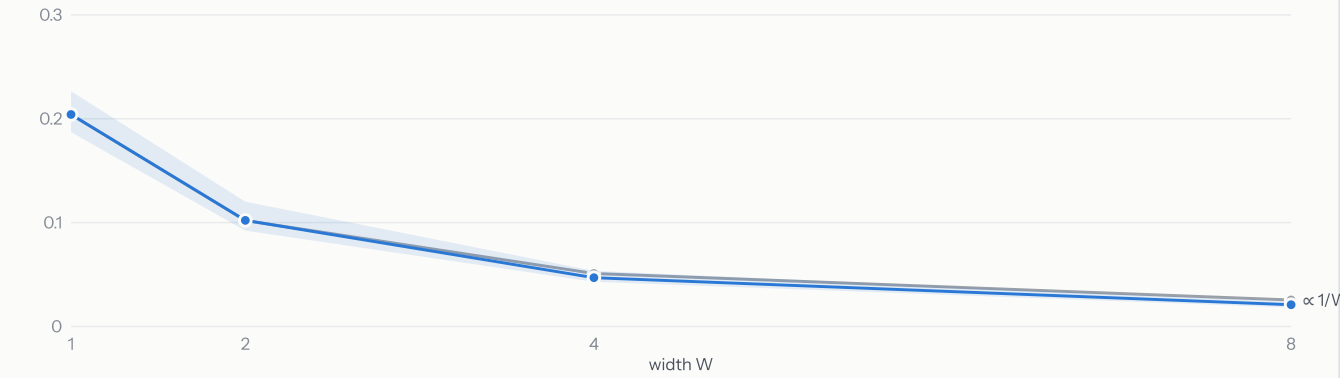
Change in P(reachable) on the untouched reachable branches after a single-branch edit. Above zero = they became more reachable.



Mean over seeds; band = seed range. Plotted: patched − unpatched P(reachable), averaged over the other reachable branches' nodes. Above zero = the untouched branches became more reachable.

Attention budget per reachable branch

Exploratory, discovery graphs, all three seeds: layer-1 attention mass from latent positions 1–3 onto one branch's edges, minus the same quantity for a decoy branch.



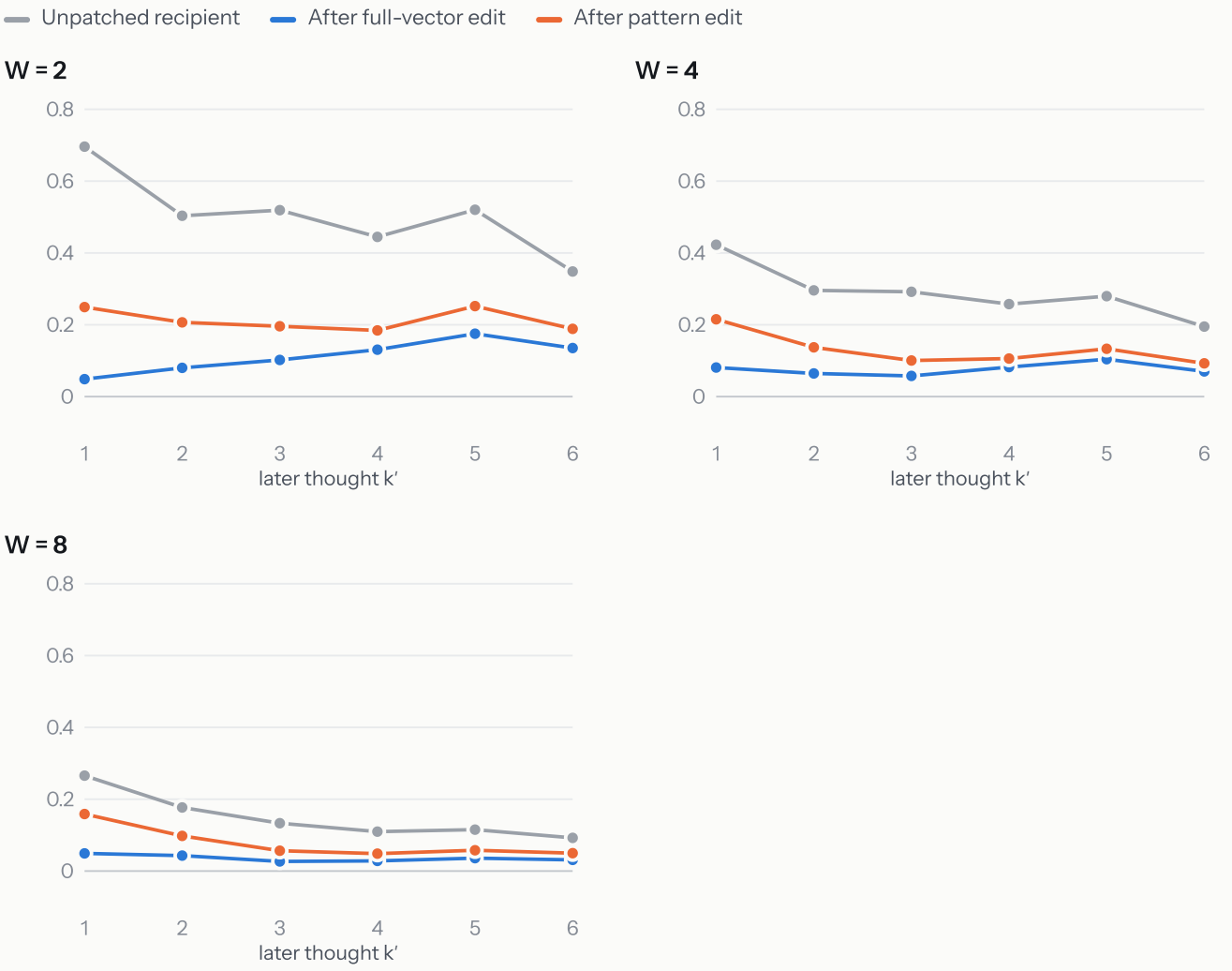
Per-branch excess attention (reachable − decoy branch), layer 1, averaged over latent positions 1–3. Grey line: 1/W scaled to the W = 1 value. Mean over seeds; band = seed range.

The edited branch partly comes back

Tracing the edited branch through later thoughts shows why single-site edits are modest. Immediately after a full-vector edit, the probe no longer separates the edited branch from decoys. In two seeds the branch then partly reappears over the next steps, because the graph and the cached early positions are still there to recompute it from. In the third seed it stays suppressed. A null effect at one interface therefore would not have shown the branch was unused, and the positive effects here are lower bounds on what the state carries.

Probe margin of the edited branch in later thoughts

Edited-branch minus decoy probe logit, unpatched vs after the edit at the frozen site. Mean over seeds.



Margin = mean probe logit of the edited branch's nodes minus decoy nodes, per later step. Seeds with k* = 0 contribute from step 1, the seed with k* = 1 from step 2.

When the target is known in advance

When the target is given before the latent steps, the thoughts become target-conditioned. Editing the target's own branch moves the answer strongly (0.14–0.74 depending on seed) while edits to the other branches barely move it (≤ 0.03 at all but one seed-width cell). The delayed-format pattern subspace is nearly inert here, so a target-conditioned state encodes the branches along different directions. This is conventional reachability, and it shows branch-level edits matter in ordinary use. It does not by itself show simultaneous alternatives, since only one branch is relevant to the answer.

W	seed 0: target-branch ΔP · other-branch |ΔP| · pattern ΔP	seed 1: target·
1	+0.156 · – · +0.001	
2	+0.175 · 0.002 · -0.000	
3	+0.152 · 0.001 · -0.000	
4	+0.167 · 0.001 · -0.000	

W	seed 0: target-branch ΔP · other-branch |ΔP| · pattern ΔP	seed 1: target·
5	+0.154 · 0.001 · -0.000	
6	+0.159 · 0.001 · -0.000	
7	+0.157 · 0.001 · +0.000	
8	+0.138 · 0.001 · -0.000	

PART II · FOLLOW-UP STUDY, 24 SEP 2026

Why do branch edits weaken with width?

Part I left the mechanism behind the width effect open. Your handoff plan named three candidate explanations. *Dilution*: a smaller per-branch signal. *Interference*: overlapping branch directions. *Recovery*: the edited branch being recomputed from retained information. The follow-up tested each with a discriminating experiment, after a preflight. The preflight reproduced all 862 published point estimates exactly from the saved checkpoint and found no family overlap between splits. It also flagged that any W = 1 edit involves an untrained width-0 state, so W = 1 is now treated as a separate distribution-shift test. Predictions were written down before any fresh graph was drawn; the scorecard below shows which held.

	prediction	seed 0	seed 0 @k=0	seed 1	seed 2	pause 0	pause 1	pause 2
P1	Route shift: latent-pathway share falls and query-graph share rises from W = 2 to 8	pass	pass	pass	pass	n/a	n/a	n/a
P1b	Pause models resolve ≥ 85% at query time	n/a	n/a	n/a	n/a	pass	fail	fail
P2	Scaling the edit to 1.5× at W = 8 does not reach the W = 2 effect	pass	pass	pass	pass	n/a	n/a	n/a
P3	At matched size, pattern directions beat probe filters	pass	pass	pass	pass	n/a	n/a	n/a
P4	Swaps cancel signed collateral but not its magnitude	fail	fail	fail	pass	n/a	n/a	n/a
P5	Inserted branch follows the recipient continuation (CI > 0)	pass	fail	pass	pass	n/a	n/a	n/a
P6	Boosting one reachable branch lowers the others; decoy/all-edge boosts ≪	pass	pass	fail	pass	n/a	n/a	n/a
P7	Restoring total gain does not restore edits	pass	pass	pass	pass	n/a	n/a	n/a
P8	Local geometry adds < 0.05 held-out R^2	fail	n/a	pass	fail	n/a	n/a	n/a
P9	Composed edits: both follow, single edits specific, sub-additive	pass	n/a	pass	pass	n/a	n/a	n/a
P10	Per-branch amplitude ∝ 1/√W fits best	fail	n/a	pass	pass	n/a	n/a	n/a

	prediction	seed 0	seed 0 @k=0	seed 1	seed 2	pause 0	pause 1	pause 2
P14	Erasing / doubling a branch's own component in the thought lowers / raises its deep nodes	pass	n/a	pass	pass	n/a	n/a	n/a

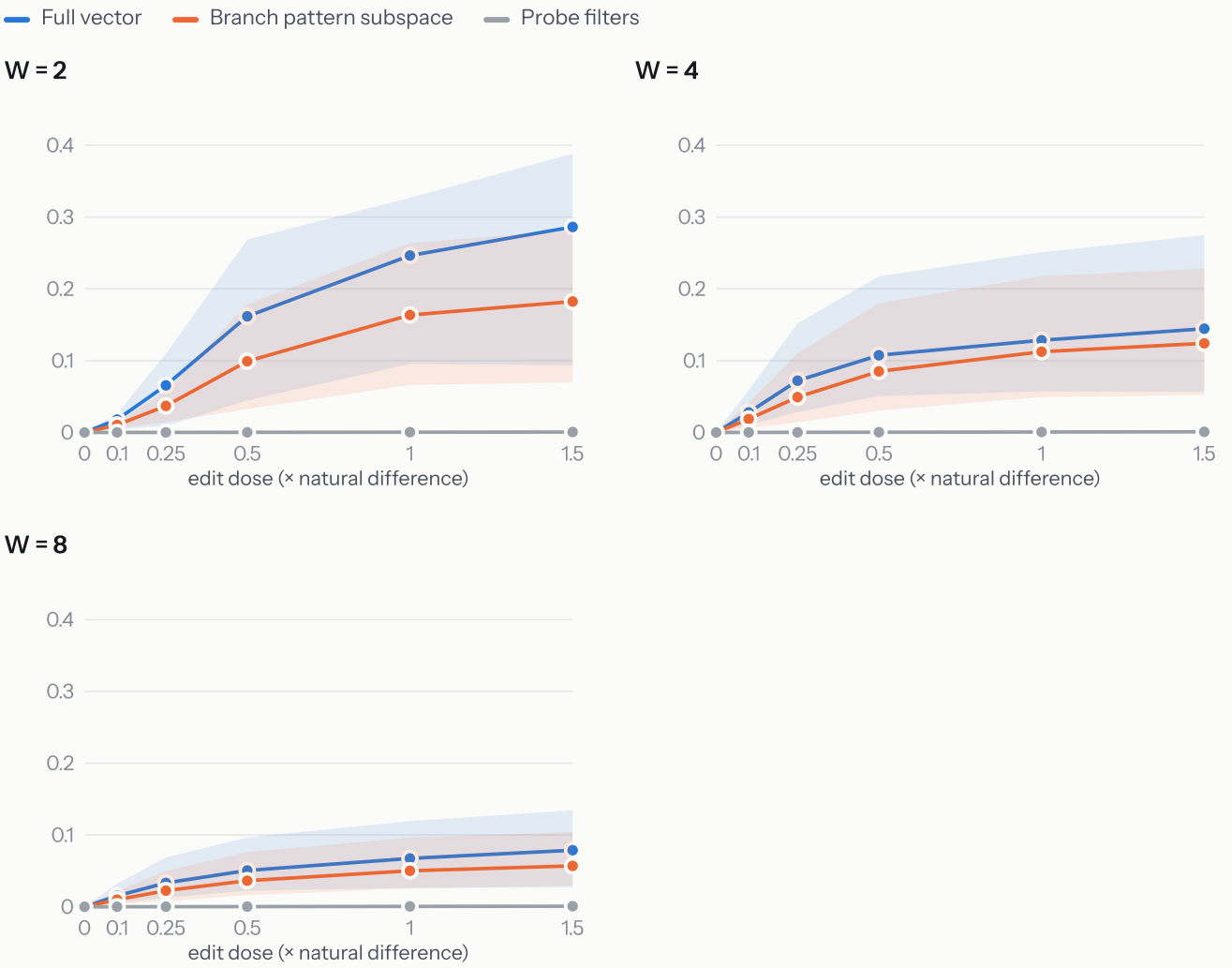
Calibration: more dose does not rescue high width

If edits weakened only because the per-branch signal is smaller, turning the edit up should restore them. It does not. At every width the full-vector effect saturates or grows slowly past the natural dose (1.0), and at eight branches no seed reaches its two-branch effect even at 1.5× (P2). As a fraction of the clean counterfactual effect (actually removing the branch), a thought edit recovers 26% at W = 2 and 11% at W = 8 (mean over seeds).

The probe-filter subspace holds only 4–6% of the donor difference's norm, so a tiny effect alone would prove nothing. The comparison that matters is at matched perturbation size. The branch-pattern edit at dose 0.1 and the filter edit at dose 1.5 have similar norms, and the pattern edit moves the branch 6–57× more in every seed (P3). Direction, not size, is what the probe gets wrong.

Targeted effect of a single-branch removal as the edit is scaled

Thought edit at each seed's frozen site; dose multiplies the (projected) donor difference. Fresh confirmation graphs; mean over seeds, band = seed range.



Clean counterfactual gap (actual removal) for reference: W = 2: 0.94, W = 4: 0.79, W = 8: 0.65. Insertion edits show a threshold instead: almost nothing below dose 0.5, then a jump (tables in results/).

Where each branch's effect travels

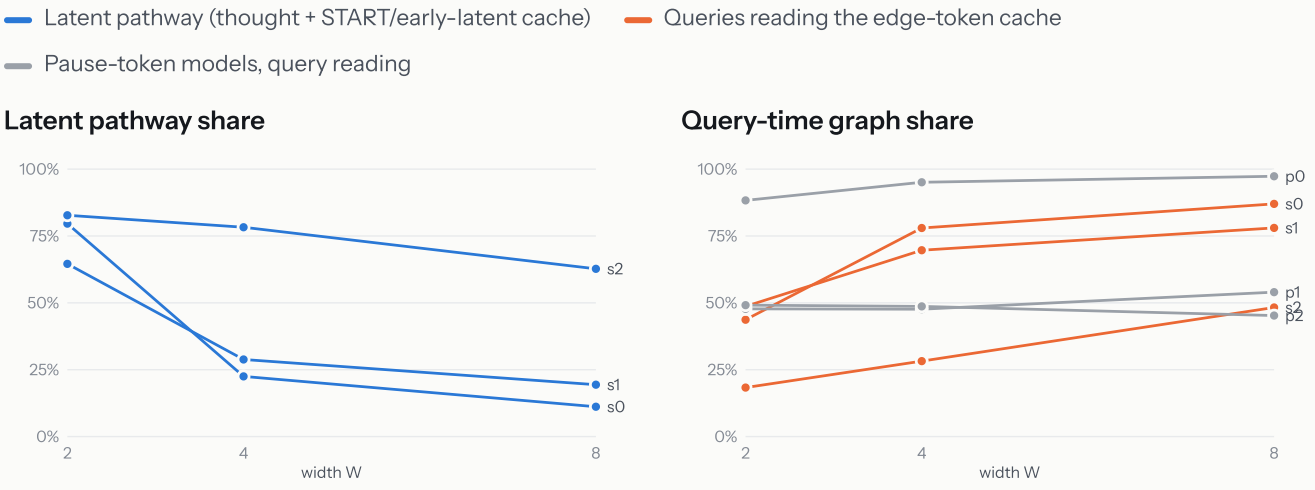
The state an edit can touch is more than one vector. For a single-branch removal we patched donor values into one route at a time. The routes were: the thought; the START and early-latent cache entries that later positions read; the edge-token cache as read by later latent steps; and the same edge-token cache as read by the queries. The shares add up. Editing the latent pathway plus the queries' view of the graph reproduces the whole clean effect at every width, in every seed.

The shares move with width (P1). At two branches most of a branch's effect travels through the latent pathway; at eight, most of it is resolved by queries reading the graph tokens directly. The breakdown by node distance below shows what that average hides. One-hop nodes are always resolved at query time, and nodes four or more hops away always through the latent pathway. What moves is the boundary, since two- and three-hop nodes switch from latent to query-time resolution at higher width. The amount of effect on deep nodes also shrinks, because the model itself becomes less decisive about them. The one competent pause seed resolves it that way at every width; a pause seed that never

learned multi-hop reachability keeps about half of it in START's cache instead (P1b, partial). Editing the latent cache alongside the thought also stops the edited branch from reappearing in later thoughts. That is the recovery the plan asked to test causally, and it runs through the retained START and early-latent entries.

Share of the clean counterfactual effect carried by each route

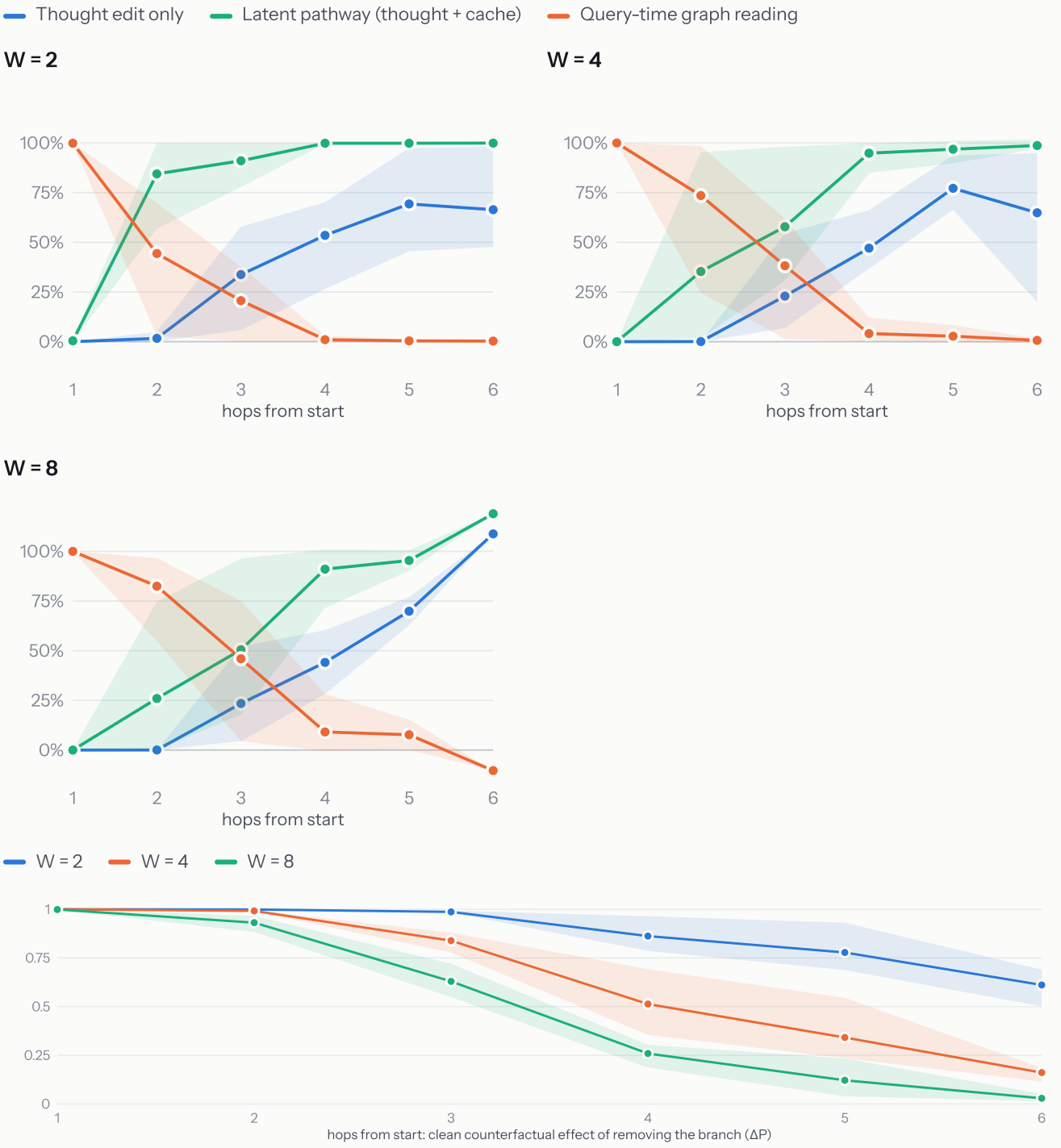
Removing one branch for real changes its nodes' P(reachable) by the clean gap. Each route is edited alone (donor values patched into that route only); shares are fractions of the clean gap. Fresh graphs.



One line per seed (labelled at the right). Latent pathway = thought edit plus the donor's START..START+k cache entries. Query-time graph = donor edge-token cache visible to the queries only. Patching graph tokens changes the effective input, so these shares locate where the difference is read, not where it is stored.

By node distance: what a thought edit reaches, and how much effect there is to reach

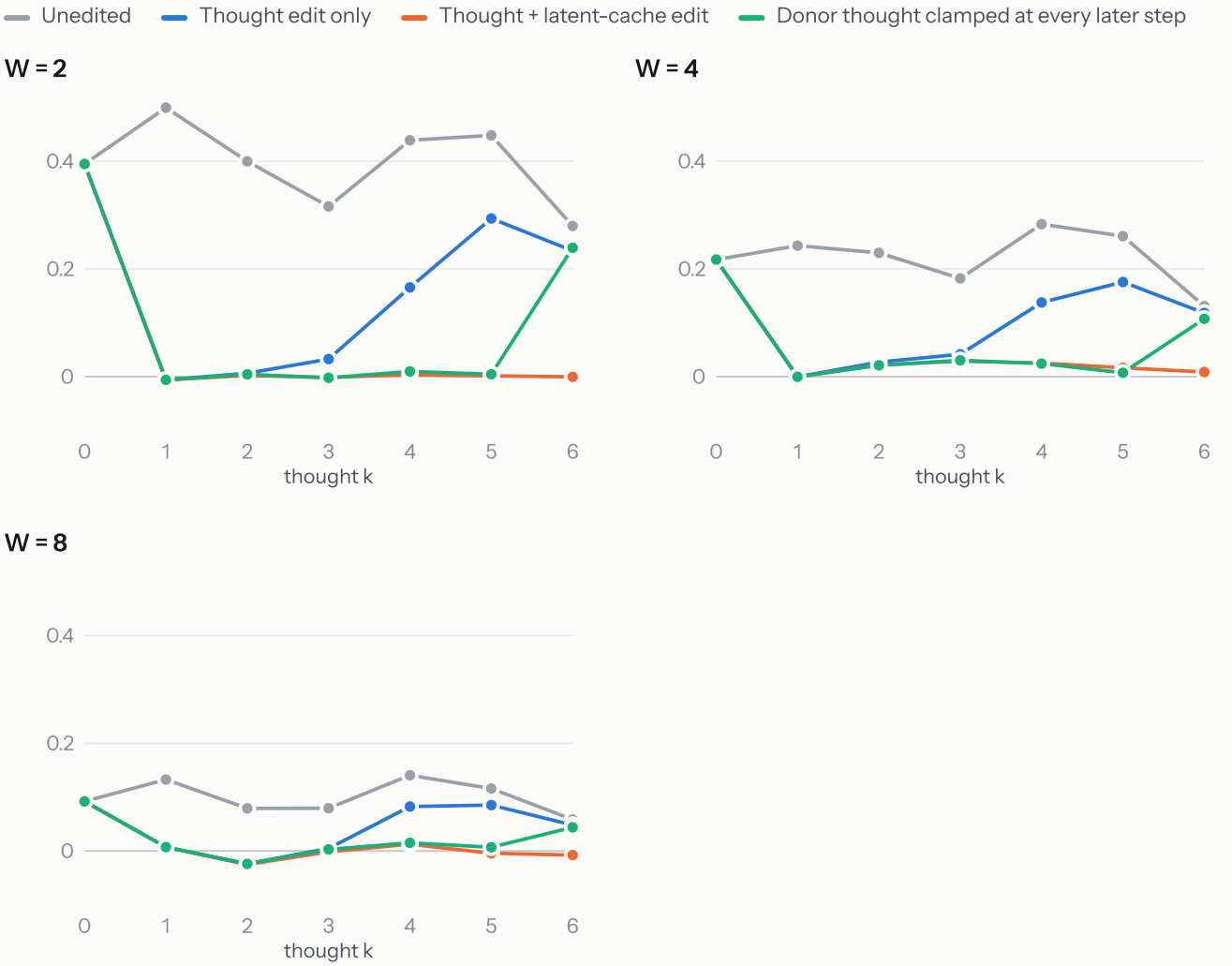
Post-hoc breakdown of the same fresh graphs. Top: share of the clean effect carried by the thought edit alone, the latent pathway, and query-time graph reading, by distance of the queried node. Bottom: the clean counterfactual effect itself. Mean over seeds; band = seed range.



Shares are shown only where the clean effect exceeds 0.05. Shares can exceed 100% or fall below 0 when routes overlap. This breakdown was not pre-registered; it re-analyses the confirmation graphs (the aggregate numbers reproduce exactly).

The edited branch comes back unless its cache route is also edited

Probe margin of the edited branch (edited-branch minus decoy logit) at each thought, seed 0 at its frozen site k* = 1; fresh graphs.



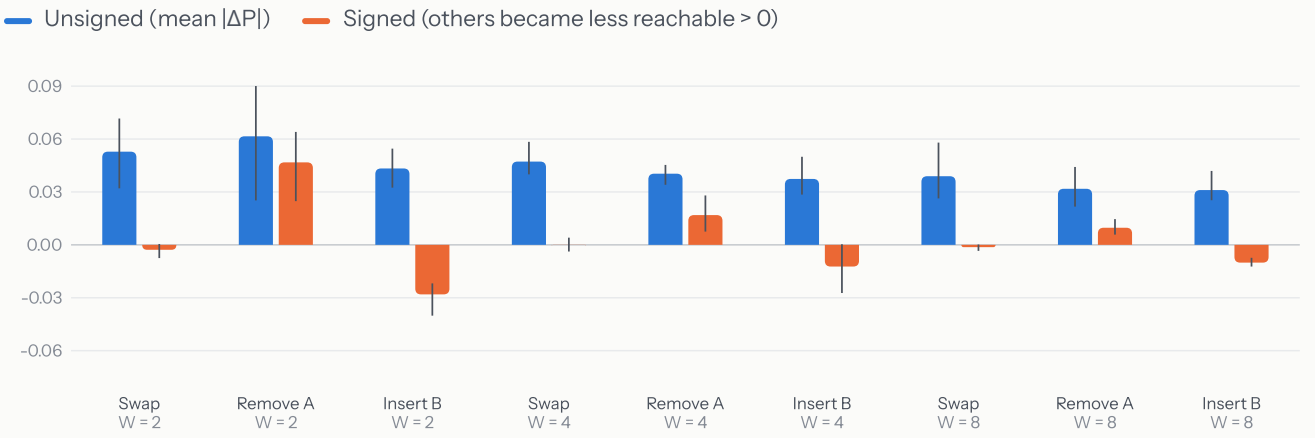
Seed 0 (k* = 1). After a thought-only edit the branch disappears from thoughts 2–3 and returns by thoughts 4–6. Adding the donor's START and LAT_1 cache entries prevents the return. Clamping the donor thought at every later step suppresses it until the final, unclamped thought. Other seeds: see results/followup_confirm.json.

Width-preserving swaps

A removal changes the number of active branches; a swap does not. If collateral came from a shared normaliser that depends on the active count, swaps should remove it. They remove its *sign*: removal makes other branches look more reachable, insertion less, and a swap sits at zero in every seed and width. But unsigned collateral on the untouched branches stays about the size of a removal's at W = 4 and 8, and only shrinks at W = 2 in some seeds (P4, partial). So active-count normalisation explains the direction of the collateral, not most of its size. The clean counterfactuals show similar cross-branch changes: actually removing a branch moves the model's answers about the other branches by 0.03–0.08. Some interference is part of how the model computes, not an artefact of the edit.

Collateral on untouched branches: swap vs removal vs insertion

Full-vector edit at dose 1. Swap = one reachable and one unreachable branch exchange hubs (width unchanged). Mean over seeds; whiskers = seed range. Collateral excludes both swapped branches.



Signed collateral plotted as (patched − unpatched) P(reachable) on untouched branches: positive = they became more reachable (after a removal), negative = less (after an insertion). Insertion at W = 8 creates a width-9 donor, outside the training range.

What an edit changes: conflicting continuations

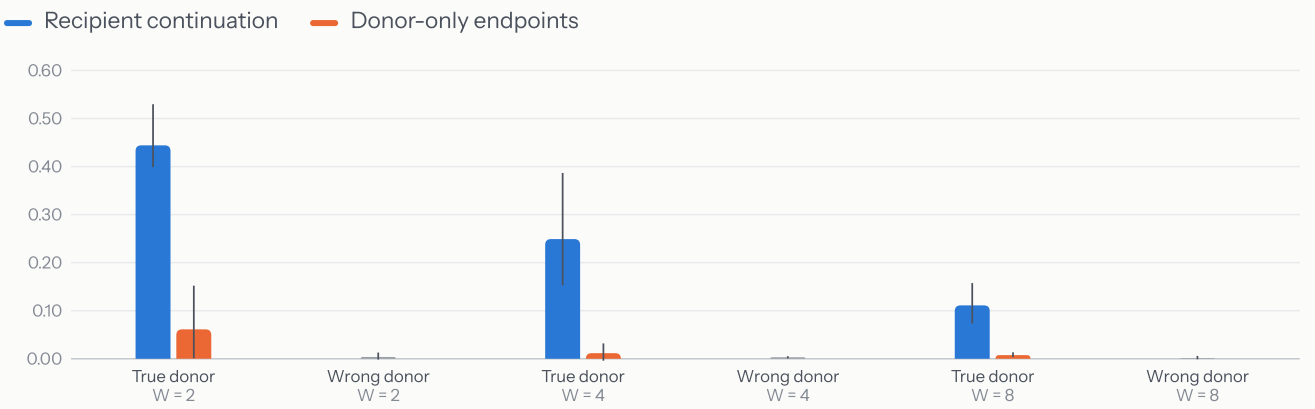
This test asks what an edit transfers. The recipient graph and the donor graph agree about branch B's upper part but disagree about what lies below depth two. There, a subtree is swapped with one from a decoy branch. We insert B's state from the donor and ask which future the model computes. If the edit transfers a branch state, the recipient's own continuation should become reachable. If it transfers the donor's pre-computed answers, the donor's endpoints should light up instead. In every seed, at every frozen site and width, the recipient continuation wins (P5): its CI lies above zero and donor endpoints barely move. A wrong-branch donor does nothing. Two branches edited at once each follow their own recipient continuation, and a single edit leaves the other branch alone (P9). The joint edit is slightly sub-additive, as competition predicts.

Two limits. The branch-pattern subspace, which carries removals well, transfers insertions poorly, so the "on" and "off" states of a branch are not mirror images in the state. And at seed 0's non-frozen common site ($k = 0$), edits at $W \geq 4$ are too weak for the test to discriminate.

An inserted branch continues with the recipient's graph, not the donor's

Donor and recipient disagree about what lies below depth 2 of branch B. ΔP(reachable) on the recipient continuation (should

rise) and the donor-only endpoints (should not). Fresh graphs, mean over seeds.



Depth j = 2 shown; j = 3 agrees (tables in results/). For the wrong-branch donor the bar is the follow index (recipient − donor endpoints). Natural insertion raises the recipient continuation by 0.88 / 0.62 / 0.40 at W = 2 / 4 / 8.

seed	W	joint edit: follow B1 / B2	single edit: other branch	joint - singl
seed 0	2	+0.251 / +0.257	-0.000 / +0.008	+0.0
seed 0	4	+0.141 / +0.164	+0.001 / +0.000	-0.0
seed 0	8	+0.068 / +0.089	+0.004 / +0.005	-0.0
seed 1	2	+0.316 / +0.271	-0.008 / -0.002	-0.2
seed 1	4	+0.162 / +0.163	+0.007 / +0.005	-0.0
seed 1	8	+0.079 / +0.083	+0.004 / -0.004	-0.0
seed 2	2	+0.366 / +0.435	+0.008 / +0.001	-0.1
seed 2	4	+0.395 / +0.405	+0.005 / +0.009	-0.0
seed 2	8	+0.178 / +0.202	+0.011 / +0.006	-0.0

A normalised, relative code

Two direct manipulations locate the competition. Adding a logit bias to the layer-1 attention onto one reachable branch's edges raises that branch and lowers the others, in proportion to the boost. The same bias on a decoy branch's edges, or on every edge at once, barely moves the reachable branches (P6 holds in two seeds; in seed 1 a uniform boost also lifts them slightly). What matters is a branch's share *relative to the other reachable branches*, not how much attention reaches edges overall. Consistently, restoring the reachable branches' total attention at W = 4 and 8 to its W = 2 level leaves edit effects unchanged (P7).

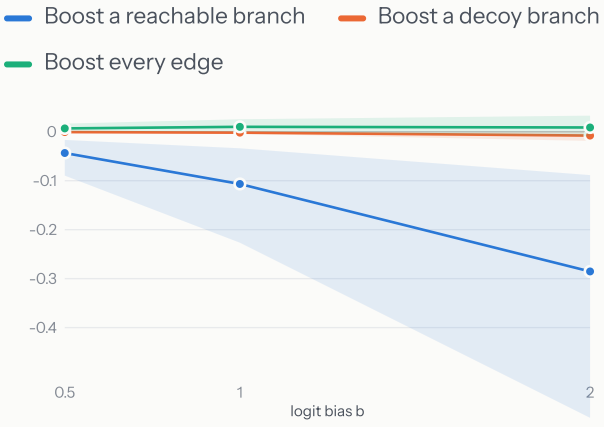
The natural branch signal in the thought, $|h(\text{donor}) - h(\text{recipient})|$, falls with width as $W^{-\beta}$, with β = 0.87, 0.61, 0.69 on fresh graphs. A LayerNorm-normalised superposition of roughly orthogonal branch codes, each taking about a 1/W attention share, predicts $1/\sqrt{W}$. That model has the lowest held-out error in two seeds; seed 0 sits closer to 1/W (P10, partial). A shared, normalised code explains why each branch's signal weakens with width, and with it the model's own counterfactual sensitivity on deep nodes. Together with the inward move of the latent/query boundary, that accounts for the weaker branch-averaged edits.

The same logic applies inside the thought itself (P14, pre-registered as addendum B). With no donor at all, we rescaled one reachable branch's own pattern component of the saved thought. Erasing it lowers that branch's nodes three or more hops away, and doubling it raises them, in every seed. The same operation on a random subspace of equal rank does almost nothing. Doubling one branch also lowers the other reachable branches in 2 of 3 seeds at W = 4, so competition is visible in the thought, not only in the attention that builds it.

seed	W	erase own: ΔP (≥ 3 hops)	double own: ΔP (≥ 3 hops)	double own: ΔP othe
seed 0	4	-0.074	+0.031	
seed 0	8	-0.054	+0.019	
seed 1	4	-0.103	+0.069	
seed 1	8	-0.155	+0.043	
seed 2	4	-0.322	+0.052	
seed 2	8	-0.163	+0.094	

Boosting attention onto one branch

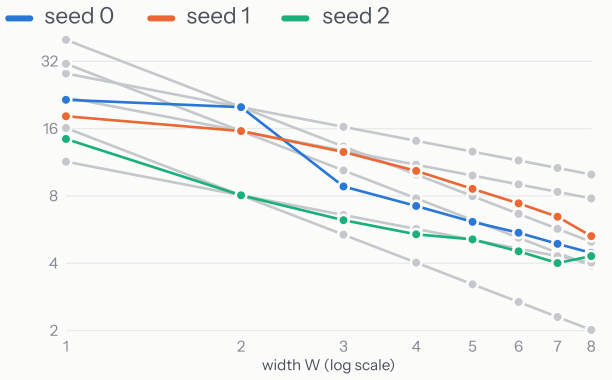
Layer-1 logit bias on one branch's edge keys (START and latent rows). Effect on the *other* reachable branches; W = 4, mean over recurrent seeds.



ΔP(reachable) on the untouched reachable branches. The boosted reachable branch itself rises by 0.092 at b = 1.

Branch-signal amplitude in the thought

|h(donor) − h(recipient)| for single-branch removal, log–log against width, fresh graphs. Grey lines: 1/√W and 1/W through each seed's W = 2 value.



Held-out MSE on log amplitude (fresh graphs, intercept fitted on discovery): seed 0: 1/√W 0.156, 1/W 0.103, constant 0.426; seed 1: 1/√W 0.082, 1/W 0.141, constant 0.240; seed 2: 1/√W 0.269, 1/W 0.296, constant 0.459. Seed 0's jump between W = 2 and 3 reflects its later edit site (k* = 1).

Local controllability

Following the plan's second approach, we linearised the forked query logits around the saved thought (Jacobian J, projected onto the shared 20-dimensional donor-difference subspace). For each branch we computed its response gain and the fraction of its response direction not shared with the other branches (ρ = ‖p‖/‖a‖, the part a perfectly selective edit could use). We then asked whether this geometry predicts held-out edit outcomes better than width, donor norm and clean confidence alone. For targeted effects it does, modestly, in two seeds (+0.07 and +0.11 R^2; P8 falsified there). For collateral it adds nothing. The linearisation itself is poor at natural edit sizes, with predicted-vs-observed

correlations of 0.15–0.31 and magnitudes overestimated. So the geometry is a weak, partial predictor, not a mechanism.

seed	targeted R^2: baseline	+ geometry	collateral R^2: baseline	+ geometry	lin
seed 0	0.24	0.31	0.23	0.24	
seed 1	0.30	0.31	0.11	0.11	
seed 2	0.39	0.50	0.05	0.07	

Replicating the controls

Part I's controls were single seeds. The replications replay each recurrent seed's exact curriculum schedule. They overturn both of Part I's single-seed control conclusions. Some control seeds never pass two hops, while one no-latent seed solves up to five hops through the prefix alone; only six-hop queries separate every recurrent seed from every control. At the cache interface, where a pause model keeps a branch depends on what it learned. The competent pause seed resolves branches at query time from the graph tokens; a pause seed limited to two hops keeps about half of the effect in START's cache.

model	delayed bal. acc	upfront	2 hops*	3*	4*	5*	6*
recurrent seed 0	90%	85%	100%	93%	75%	67%	66%
recurrent seed 1	92%	87%	100%	96%	81%	67%	66%
recurrent seed 2	93%	90%	99%	92%	83%	79%	59%
pause seed 0	91%	81%	100%	99%	81%	56%	53%
pause seed 1	71%	68%	97%	52%	51%	52%	50%
pause seed 2	72%	71%	99%	51%	52%	53%	55%
no-latent seed 0	70%	67%	93%	54%	54%	55%	55%
no-latent seed 1	91%	71%	100%	98%	86%	54%	52%
no-latent seed 2	93%	86%	100%	99%	88%	67%	56%

* depth-matched balanced accuracy (reachable vs same-depth decoy nodes; chance 50%).

Part III: ablations, held-out seeds, and test-time search

Part III asks whether the Part II account survives changes it should be sensitive to. It was pre-registered before any of its models was trained (`docs/PROTOCOL3.md`). Three architecture variants each change one element the account depends on: a wider thought (V1), a deeper network (V2), and recurrence through the raw residual instead of the final LayerNorm (V3). Three further seeds, trained after the predictions they are scored against were frozen, test whether Part II's predictions hold on seeds that played no part in forming them. Two exploratory findings from the same period get their own sections: in the upfront format queries never read the graph, and one seed learned a search that keeps working when run for more steps than it was trained with. That second finding was turned into a prospective prediction (P15) before the new models were evaluated.

All Part III analyses reuse the frozen Part II pipeline on fresh graphs (300 per width). Training on this rig proved fragile. Two host crashes killed runs in progress, one GPU dropped off the bus at the end, and about half of all gated training attempts stalled at the first curriculum stage; those were restarted under a rule written before their evaluation (details). Several of the new runs still failed the competence gate and are reported as training failures.

	prediction	model	result	
P11	Wide thought (d = 512): width effect persists — targeted W8/W2 ≤ 0.5 and per-branch attention excess W8/W2 ≤ 0.5	V1 wide (d = 512)	targeted 0.58, attention 0.24	fail
P12	Deep network (4 layers): query-time share at W = 4 above the reference minimum; 3-hop nodes mostly query-resolved	V2 deep (4 layers)	Gq 17% vs ref. min 28%; 3-hop Gq 0%	fail
P13	Raw-residual feedback: amplitude exponent β leaves [0.62, 0.87] (toward 1); competition persists	V3 raw feedback	β = 0.97; boosting one branch lowers others: yes	pass
C	Held-out seed: every Part II prediction that held for all models (P1, P2, P3, P5, P7, P9, P14) holds again	seed 3 (held out)	6 of 7 hold	fail
D	Held-out seed: every Part II prediction that held for all models (P1, P2, P3, P5, P7, P9, P14) holds again	seed 4 (held out)	3 of 7 hold (failed competence gate)	not scored
I1	Held-out seed: every Part II prediction that held for all models (P1, P2, P3, P5, P7, P9, P14) holds again	seed 5 (held out)	2 of 7 hold	fail
P15	A model extrapolates to unseen chain lengths iff extra steps help its depth-6 accuracy (≥ +0.05, K′ 6 → 8)	new recurrent models	2 of 3 agree; not scored (failed competence gate): seed 4 (held out)	fail
P16	No new model whose layer-1 sweep reaches depth 4 within six steps extrapolates	new recurrent models	seed 3 (held out): E = 5, no extrapolation; V1 wide (d = 512): E = 3, no extrapolation; V2 deep (4 layers): E = 4, no extrapolation; not scored (failed competence gate): seed 4 (held out)	pass
H	Second wide-thought seed: targeted W8/W2 > 0.5 again → the wider thought causes the weaker decline; 0.18–0.37 → V1's run	V1 wide, seed 1	failed competence gate (72% after four attempts): unresolved	not scored
G	Training budget (exploratory): K = 10 should not extrapolate; K = 4 should either extrapolate or read the graph at query time	budget K = 4, budget K = 10	K = 10: failed competence gate (not scored); K = 4: extrapolates, query share W4 20%	pass
I2	Second K = 4 seed: if competent, it extrapolates or reads the graph at query time	budget K = 4, seed 1	failed competence gate	not scored
P17	Holding the step embedding at step 2 lets a non-extrapolating model reach distance 7 (≥ 60%, lower bound > 55%)	V1 wide (d = 512); budget K = 8	V1: 57% [55%, 59%]; K = 8: 58% [56%, 60%]	fail
J1	K = 4, seed 2 (stricter gate): extrapolates or reads the graph at query time	budget K = 4, seed 2	distance 7 at 12 steps 0.71; query share W4 21%	pass

	prediction	model	result	
J2	K = 8: sweep reaches depth 4 within six steps and no extrapolation at 16 steps	budget K = 8	E = 4; distance 7 at 16 steps 0.56	pass
P18	No step embedding: the model extrapolates (distance 7 ≥ 60% at 12 steps)	no step embedding	distance 7 at 12 steps 0.63 [0.61, 0.65]; E = 3	pass

Changing one piece of the architecture

Each variant keeps the seed-0 recipe (scaffold, gated curriculum, 30k steps) and changes one thing. The table shows the quantities the Part II account rests on, next to the three reference seeds.

V3, raw-residual feedback. Coconut feeds the final-LayerNorm output back as the next input. V3 feeds the residual stream before that LayerNorm. It trains to 88% delayed accuracy. The Part II account attributes the per-branch amplitude law to renormalisation of the shared state, so it predicted that β would move toward 1 (per-branch shares of about 1/W no longer rescaled), while competition, which arises in attention, would stay. Both held: β = 0.97, outside the reference range 0.62–0.87, and boosting one reachable branch's attention still lowers the others (P13). Without the renormalisation, V3's branch edits are also larger (targeted 0.48 at W = 2 against 0.10–0.33 in the reference seeds).

V1 and V2 test the other two candidate limits. If the width effect came from superposition capacity, doubling the width of the thought should weaken it (P11). If it came from the prefix's limited depth of computation, extra layers should move more nodes to query-time resolution (P12).

V1, a wider thought (d = 512). The normalisation account predicted that the width effect would survive. Half of that held: per-branch attention still falls about as 1/W (W8/W2 = 0.24). The targeted effect, though, declines much less than in any d = 256 seed (W8/W2 = 0.58 against 0.18–0.37), so P11 fails. The direction is the one a capacity account predicted. Its other prediction, lower collateral than seed 0, does not hold (0.04 vs 0.02 at W = 8). V1 also keeps most of each branch's effect in the latent pathway at every width, like seeds 2 and 3. With one wide run we cannot say whether this is the extra width or the run.

V2, a deeper network (4 layers). Its first run was lost at step 23 000 in the second host crash; the rerun is the most competent model of the project (96% delayed, 91% at six hops, 100% upfront). The prediction was that extra layers would let queries read more hops directly from the graph. The opposite happened. V2's query-time share at W = 4 is 17%, below every reference seed (minimum 28%), and at three hops it is 0%. The extra layers went into the latent pathway, which keeps 78% of each branch's effect even at eight branches. Its targeted effect also declines less with width than in any d = 256 seed (W8/W2 = 0.55), much like V1. Across all the competent models, the strong route shift of Part II turns out to be specific to seeds 0 and 1. In seed 2, seed 3, V1, V2 and the K = 4 run, the latent pathway still carries 63–78% of a branch's effect at eight branches, and in V3 about half.

model	delayed acc.	6-hop matched	targeted W2	targeted W8	W8 / W2	latent shar
seed 0	90%	66%	0.10	0.03	0.28	
seed 1	92%	66%	0.32	0.06	0.18	
seed 2	93%	59%	0.33	0.12	0.37	
seed 3 (held out)	93%	86%	0.56	0.19	0.33	
seed 4 (held out)	82%	53%	-0.00	-0.00	0.05	

model	delayed acc.	6-hop matched	targeted W2	targeted W8	W8 / W2	latent shar
seed 5 (held out)	89%	54%	-0.00	-0.00	0.94	
V1 wide (d = 512)	92%	59%	0.23	0.13	0.58	
V1 wide, seed 1	72%	51%	–	–	–	
V2 deep (4 layers)	96%	91%	0.57	0.31	0.55	
V3 raw feedback	88%	57%	0.48	0.13	0.26	
budget K = 4	93%	59%	0.55	0.12	0.22	
budget K = 4, seed 1	69%	61%	0.00	-0.00	-0.02	
budget K = 10	84%	53%	0.01	0.01	0.66	

Held-out seeds

Seed 3 was started after Part II's predictions were frozen, and seed 4 after Part III's were. Neither influenced any threshold. Each is scored with the unchanged Part II scorer. Before evaluation, we expected the seven predictions that held for every Part II model to hold again; the four mixed ones are reported without a directional expectation.

Seed 3 is the strongest model of the project: 86% depth-matched accuracy at six hops, against 59–66% for seeds 0–2. Its branch edits are large (targeted 0.56 at W = 2) and mostly stay in the latent pathway (72% at W = 8). 6 of the 7 expected predictions hold. The exception is P7, and it fails in the opposite direction from a rescue: restoring total gain slightly *lowers* the targeted effect (W = 4: 0.43 → 0.40), which is outside the frozen tolerance of 0.03. Its per-branch amplitude barely falls with width (β = 0.29), so P10 fails too. Competition inside the thought appears here as well, in three of four seeds now.

Seed 4 failed training (82% delayed accuracy) and is not scored. A third held-out seed was added in its place. **Seed 5** clears the gate (89%) but reaches only 54% at six hops, and it does not use its thought. At every candidate site a branch edit has a targeted effect below 0.0001, so the site rule picked one at random. Branch effects travel through query-time graph reading (89% at W = 4), and its first layer shows no sweep. The thought-edit predictions therefore fail for want of anything to edit: 2 of 7 expected predictions hold. The verdict stands as scored. The lesson is about the gate: overall accuracy admits a model that never uses the latent loop, as the no-latent controls already showed (91–93% overall). A depth-matched gate would have excluded it.

prediction	Part II status	Part II (all models)	seed 3 (held out)	seed 4 (held out) (failed gate)	seed 5 (held out)
P1	held for every Part II model	pass	pass	pass	pass
P2	held for every Part II model	pass	pass	fail	fail
P3	held for every Part II model	pass	pass	pass	fail
P5	held for every Part II model	fail	pass	fail	fail
P7	held for every Part II model	pass	fail	pass	pass
P9	held for every Part II model	pass	pass	fail	fail

prediction	Part II status	Part II (all models)	seed 3 (held out)	seed 4 (held out) (failed gate)	seed 5 (held out)
P14	held for every Part II model	pass	pass	fail	fail
P4	mixed in Part II (no expectation)	fail	pass	fail	fail
P6	mixed in Part II (no expectation)	fail	pass	pass	fail
P8	mixed in Part II (no expectation)	fail	pass	pass	fail
P10	mixed in Part II (no expectation)	fail	fail	fail	pass

With the target given first, queries never read the graph

Part II found that as width grows, delayed queries resolve more nodes by reading the graph tokens themselves, bypassing the thought. In the upfront format the target node comes before the latent steps and one ANSWER token reads the result. Applying the same route decomposition there (exploratory, discovery graphs) gives a clean answer in all three seeds. The ANSWER token's own reading of the graph carries none of the target branch's effect, at every width and every target distance, and the latent pathway (thought plus START, TARGET and early-latent cache) carries all of it. Seed 1's thought alone carries everything; seeds 0 and 2 split it between the thought and the early cache.

So the route shift that Part II used to explain the width effect is specific to the delayed diagnostic. When the model knows what it is looking for, it computes the answer inside the latent loop. This is also why upfront branch edits were selective in Part I.

seed	W	clean effect	thought only	latent pathway	query reads graph	delayed:
seed 0	2	0.74	22%	100%	-0%	
	4	0.66	26%	100%	-0%	
	8	0.59	27%	101%	-0%	
seed 1	2	0.75	100%	100%	0%	
	4	0.72	100%	100%	0%	
	8	0.64	100%	100%	-0%	
seed 2	2	0.90	60%	100%	0%	
	4	0.75	65%	100%	0%	
	8	0.58	77%	100%	-0%	

Two search algorithms: a forward sweep and a backward walk

Attention traces show what each latent step reads from the graph (exploratory, discovery graphs). In the delayed format every recurrent model sweeps *forward*. At each step its first layer attends to the edges of one within-branch depth across all reachable branches, leaving decoy edges at baseline, and the depth advances with the step. The models differ in speed. Seed 1 advances one depth per step and reaches the deepest trained depth at step 5, seed 0 one step later, and seed 2 advances one depth every two steps (depth 3 at step 6). The sweep is causal and clocked. Blocking one branch's edges of a given

depth at the step that reads them lowers that branch's deeper nodes (by 0.11–0.15 in seed 1, up to 0.34 in seed 2); blocking the same edges at any other step changes nothing. Width leaves the clock unchanged: the step at which each depth is read is the same at two, four and eight branches. It changes the amplitude. Seed 1's per-edge attention on the depth-1 frontier at step 1 is 0.072, 0.034 and 0.017 at W = 2, 4 and 8, so the total on the frontier stays near 0.14: a fixed attention budget shared among the branches. That is the mechanism behind the per-branch amplitude law of Part II, and behind the finding below that width limits the search's fidelity rather than its speed.

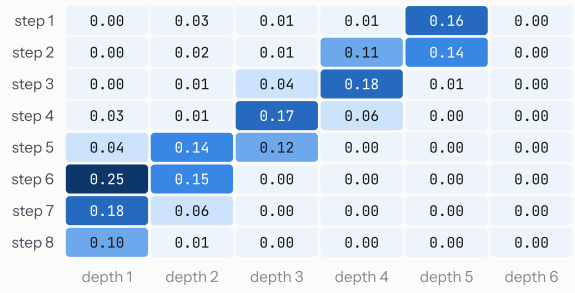
Pause-token models show no such sweep. Their latent positions cannot carry graph content into the first layer, and in the second layer two of the three pause seeds attend to the reachable hub edges equally at every step. The stepwise frontier needs the recurrent thought.

With the target given first, V3 does something else. Its attention starts at the edge into the target's parent and climbs the target's ancestry one edge per latent step until it reaches the hub edge at step 6. Whether that hub edge comes from the start or the decoy hub settles the answer. Blocking the edge the walk should read at a given step lowers the answer by up to 0.48; blocking it one step early or late, or blocking a same-depth edge elsewhere, does nothing. This backward pointer-chasing follows a single path, so it cannot depend on width. V3's target-first accuracy is 100% at every width from 1 to 8, it never needs to read the graph at answer time, and with more steps it follows chains to distance 7 (85% after sixteen steps) even though it does not extrapolate in the delayed format.

The forward sweeps suggest why seeds differ at test time. Seeds 0 and 1 finish the trained depth inside the six-step budget, and seed 0 at least then enters a state that stops the search (next sections). Seed 2's slower sweep is still running at step 6. This led to a second prospective prediction (P16): no new model whose sweep reaches depth 4 or more within six steps will extrapolate. Frontier depth reached by step 6 so far: seed 0 5, seed 1 5, seed 2 3, V3 2.

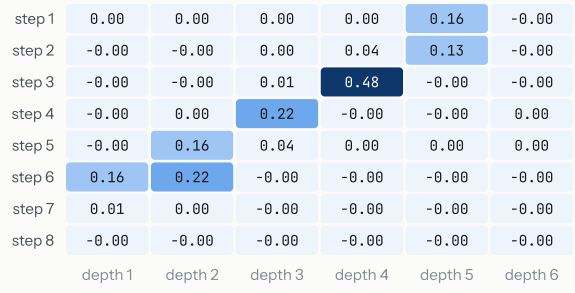
V3, target given first: attention walks back from the target

Layer-1 attention of each latent step onto the edges of the start-to-target path (path depth 1 = hub edge, 6 = edge into the target), target at distance 6, W = 4.



Blocking one edge at one step

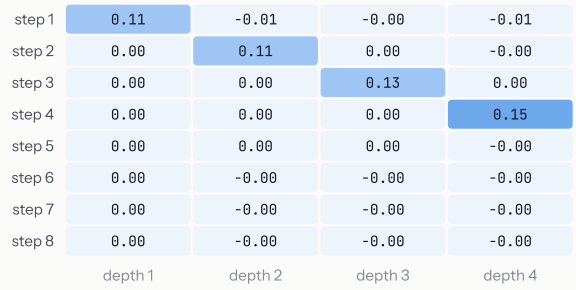
Drop in P(reachable) when layer-1 attention onto one path edge is blocked at one step. Same-depth edges of other reachable branches: |Δ| ≤ 0.004.



V3 only; 400 graphs, baseline P(reachable) 1.00. Largest effect of blocking a same-depth edge in another reachable branch: 0.004. Seeds 0 and 1 show no step structure in the target-first format; seed 2 sweeps forward instead (attention on all same-depth edges of reachable branches).

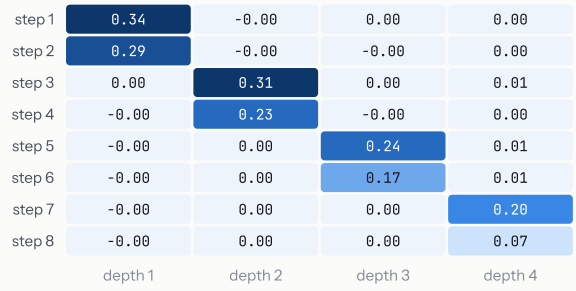
Seed 1, delayed format: one depth per step

Drop in P(reachable) of a branch's deeper nodes when its edges into depth-x nodes are blocked at one latent step (W = 4).



Seed 2, delayed format: one depth per two steps

Same test. Each depth is read over two consecutive steps, and the sweep is still running at step 6.



Seed 0 matches seed 1 shifted by one step (depth x read at step x + 1, nothing after step 5). Held-out seed 3 also advances one depth per step, and blocking the right step lowers the branch's downstream P(reachable) by up to 0.80; V1 reads depths 1–2 in two passes (steps 1–3, then 4–5). Blocking one branch's edges raises the other reachable branches by at most 0.042: the competition of Part II.

Some seeds learn a search that can run longer

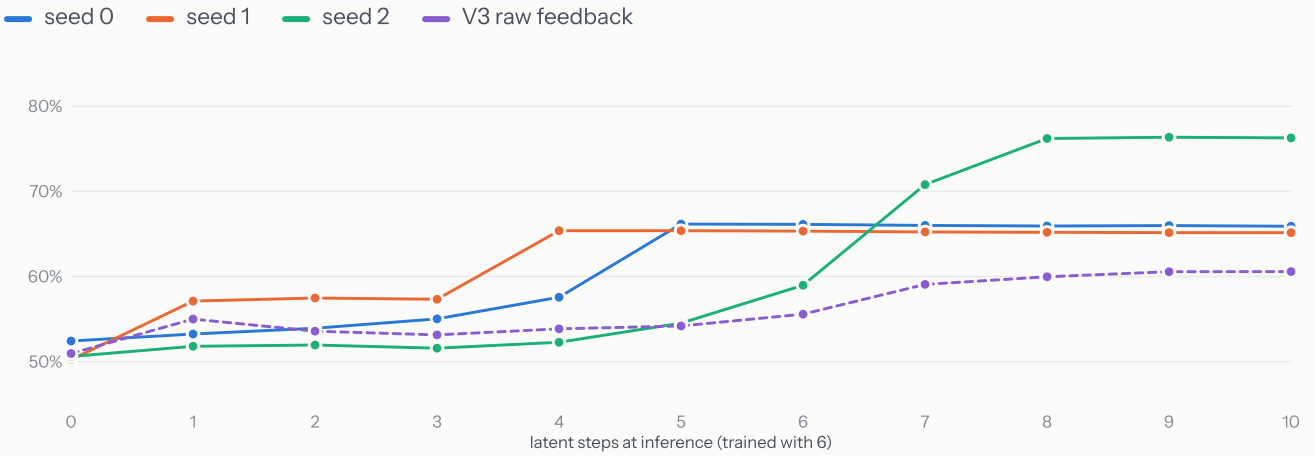
Truncating the latent budget below six steps was not a clean ablation (Part I). Extending it is. Each model ran its recurrence for up to ten steps (the step-6 embedding reused for steps 7–10) and was scored with depth-matched decoys on discovery graphs. The seeds turn out to differ in kind. Seed 2 resolves depth one step at a time, a staircase: three hops by step 2, four by step 4, five by step 6. It keeps going past its training budget, with six-hop accuracy rising from 59% at six steps to 76% at eight. Seeds 0 and 1 have resolved everything they will by step 4–6 and gain nothing from more steps. Pause-token and no-latent models gain nothing either.

Chains longer than any in training make the difference sharper. Re-hanging a decoy chain from the leaf of a reachable one puts nodes at distances 7–12, which never occur in training. Seed 2 reaches distance 7 with 71% balanced accuracy after twelve steps (chance before step 8) and distance 8 with 62%, roughly one hop per two steps. Every other model stays at chance beyond six hops however long it runs. All three seeds sweep forward step by step (previous section), but only seed 2's sweep can simply be run longer. Seeds 0 and 1 stop once their faster sweep has covered the trained range. On the training distribution the two behaviours reach similar accuracy.

Width limits this search through fidelity, not time. In seed 2 depth 6 resolves at about the same step at every width, but the level it resolves to falls with width. Extra steps bring each width to its own ceiling (depth 6: 94% at W = 2, 58% at W = 8) and never above it. That fits Part II's normalised code: each branch's share of the state shrinks with width, and each hop copies it with more loss. After seeing this we pre-registered P15. On every newly trained recurrent model, in-range gains from extra steps should predict out-of-range extrapolation. It failed. V1 gains +0.050 at six hops from two extra steps, just over the bar, yet stays at chance at distance 7. In-range gains are not a reliable sign of an open-ended loop.

Depth-6 accuracy against the number of latent steps

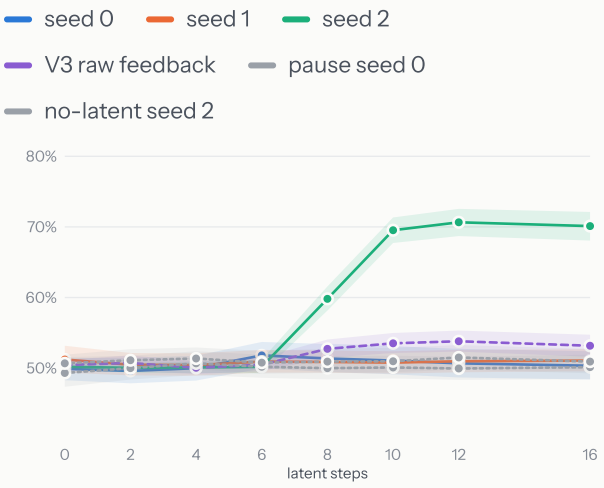
Depth-matched balanced accuracy at six hops, discovery graphs. Every model was trained with six latent steps; steps 7–10 reuse the step-6 embedding.



Dashed lines: architecture variants. Beyond six steps the model runs its recurrence longer than it ever did in training.

Chains longer than any in training

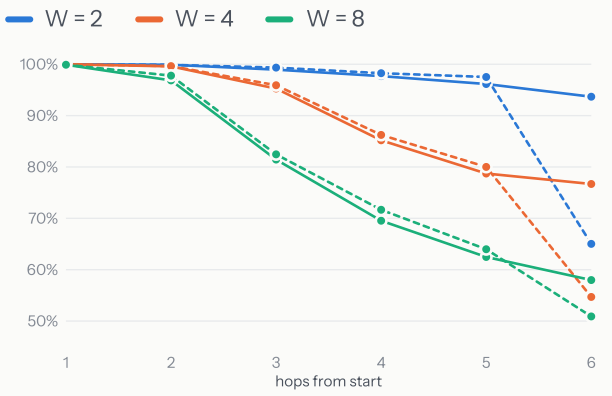
Balanced accuracy at distance 7 (never seen in training) on all-chain graphs, against latent steps.



Band: 95% bootstrap interval over graphs. Positives: a decoy chain re-hung from the leaf of a reachable chain (distances 7–12). Negatives: the same shape re-hung from a decoy leaf.

Seed 2: extra steps help at W = 2, not W = 8

Depth-matched accuracy by depth after 10 latent steps (solid) and 6 (dashed), per width.



Seed 2 only. Ten steps move depth 6 to the width's own ceiling but never raise the ceiling.

Rewinding the thought: one seed's search is halted, not incapable

Why do seeds 0 and 1 stop at six hops? Either their recurrence cannot propagate further, or something stops it. A rewind separates the two. During a run, the thought fed into step j is replaced by the model's own earlier thought while the key/value cache, which holds everything computed so far, is kept (exploratory, long-chain graphs).

In seed 2, rewinds only cost reach: the thought carries its search state, and an old thought does not fit the newer cache. In seed 0 a single rewind does the opposite. Feeding a thought from steps 1–3 into step 6 or later lifts accuracy at distances 7–9 from chance to about 62%, with no cost at shorter distances, and in a twenty-step run it reaches distance 12. Thoughts from step 4 onward do nothing. So does the same step's thought taken from a different graph. The gain is largest at two branches (balanced accuracy 0.49 → 0.66) and small at six.

Seed 0's propagation machinery therefore works past its trained depth. A first guess was that a state its thought enters around step 4 stops it. That state is not a simple, graph-independent direction, and the next section shows that the stop is keyed to the step embedding instead: shifting a late thought by the average difference between early and late thoughts does not unlock the search (far accuracy at most 0.52). Only an early thought from the same graph, combined with the propagated cache, restarts it. No rewind helps seed 1, seed 2 or V3. Held-out seed 3, whose sweep also finishes early, shows the same signature more weakly. Its best rewinds all feed back its step-3 thought and lift far accuracy from 0.51 to 0.56.

Seed 0: gain in far accuracy after one rewind

Twelve-step runs on long-chain graphs. At step j the input thought is replaced by the run's own thought from step s; the cache is kept. Cell: balanced accuracy at distances 7–9 minus the unedited run (0.51).

	s = 0	s = 1	s = 2	s = 3	s = 4	s = 5	s = 6	s = 7
inject at step 3	-0.00	-0.00						
inject at step 4	0.05	-0.00	0.00					
inject at step 5	0.07	0.06	-0.00	0.00				
inject at step 6	0.04	0.09	0.11	0.12	0.01			
inject at step 7	0.03	0.06	0.11	0.11	0.01	0.00		
inject at step 8		0.03	0.08	0.11	0.01	0.00	0.00	
inject at step 9			0.04	0.08	0.01	0.00	0.00	-0.00
inject at step 10				0.03	0.01	0.00	0.00	0.00
inject at step 11					0.01	0.00	0.00	0.00

Largest gain from any rewind in the other models: seed 1 +0.023, seed 2 -0.004, V3 +0.009. Seed 0 controls (12 and 16 steps): the thought from a different graph gives at most +0.057 (only for the encoder output, s = 0), a zero vector at most +0.042.

The stop is keyed to the step embedding

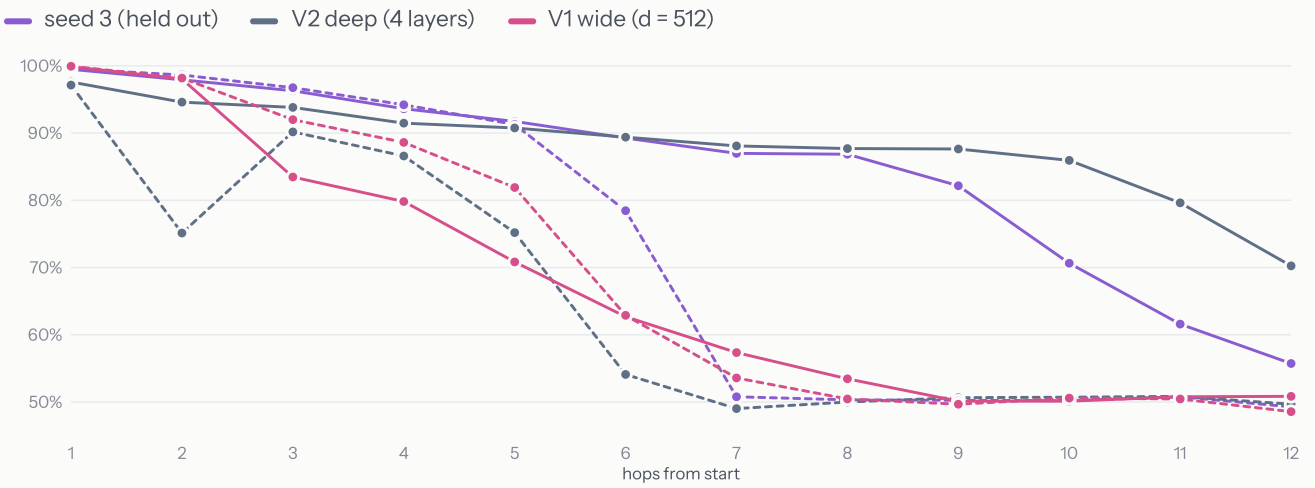
The rewinds changed the thought. A cleaner test changes only the step embedding each latent step receives, the model's sole explicit signal of how far into the loop it is, and leaves the thought and the cache alone (exploratory). Two things separate. First, the depth a step reads does not follow the step embedding. Hold the embedding at an early step, or delay it, and every model still reads depth 1, then 2, then 3, at the same steps as before. The frontier is carried by the thought. Second, the *stop* does follow the step embedding. Advance it by two steps and seed 3's sweep ends early (six-hop accuracy 0.84 → 0.51). Hold it at step 2 and the models that normally stop at six hops keep going: seed 3 reaches 87% at distance 7 and 82% at 9, and V2 88% at 7 and 86% at 10, where their normal runs are at chance.

So the propagation step these models learned works well past the depth they were trained on, and what stops them is a halt keyed to the step index, learned because every training episode ended by step 6. The attention trace below shows that the halt acts on the sweep itself. With normal step embeddings V2's first layer stops reading edges at step 6. With the embedding held at step 2 it walks

the long chain one distance per step, through distances 7–12, and stops only at the chain's end. Seed 2's slower sweep never finished early, never learned the halt, and extrapolates without help. This supersedes the reading of the rewind results as a halting state in the thought: a rewind restores early thought content but keeps the late step embeddings, and it unlocks much less. A training-level test agrees. A model trained with no step embedding at all (P18, pre-registered) has no step index to learn a halt from, and it extrapolates: 63% at distance 7 after twelve steps, still above 60% at distance 12 after sixteen, gaining about one hop per extra step. The effect of holding the clock is not universal. Seeds 0 and 1 gain less (about 60–65% out to distance 12). The two models tested only after it was pre-registered as P17, V1 and the K = 8 run, gain too little to count. Their traces show why: their sweeps do not halt sharply at a step but fade hop by hop, reaching distance 7 too weakly to matter, and a held clock cannot fix that. So there are two kinds of limit, a step-keyed halt (seed 3, V2) and per-hop attenuation (K = 8, V1), and P17 had assumed the first everywhere (distance 7: 57%, bar 60%).

Holding the step embedding at step 2 lets the search run on

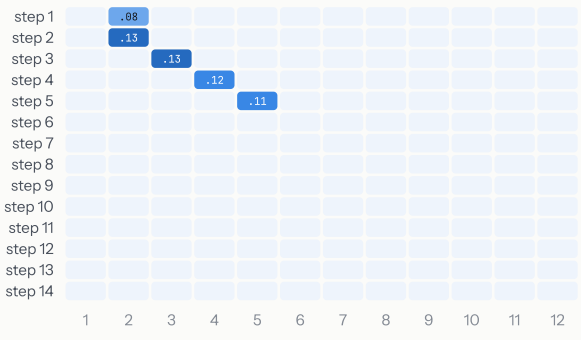
Balanced accuracy by distance on long-chain graphs after 20 latent steps: normal step embeddings (dashed) and every step fed the step-2 embedding (solid). Distances beyond 6 never occur in training.



Seeds 3 and V2 show the effect most clearly; V1 is the prospective test (P17), which failed. With the clock held at 2, seeds 0 and 1 reach 63% and 61% at distance 9 (normal: chance). Seed 2, which never stops, is unaffected and not shown.

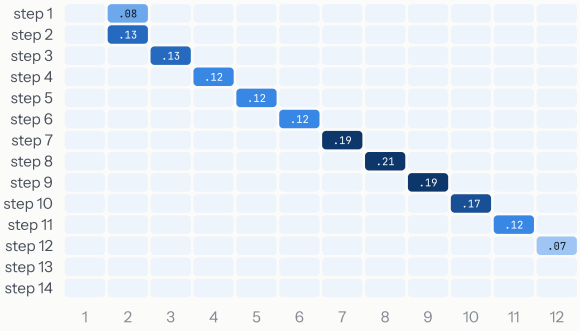
V2, normal step embeddings: the sweep halts at step 6

Layer-1 attention of each latent step on the chain edge into each distance (long-chain graphs; distances 7–12 are the re-hung chain).



V2, step embedding held at step 2: it walks to the chain's end

Same graphs and scale. One new distance per step, through distances never seen in training.



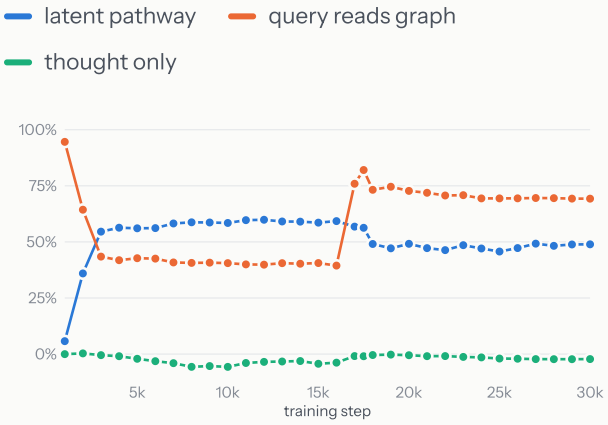
How the routes develop during training

Seed 4 saved its weights every 1 000 steps and at each curriculum advance. Each checkpoint was run through the route decomposition and the attention trace (exploratory, discovery graphs, k = 1). Seed 4 turned out to be a training failure (82% delayed accuracy, below the 85% gate), which makes its trajectory informative for a different reason. It shows what happens when the clocked sweep does not form.

At the first checkpoint, on star-shaped branches, a branch's effect travels almost entirely through query-time graph reading, and already more so at higher width (latent share 16% at W = 2, 2% at W = 8). As two-hop nodes are learned, the latent pathway grows to about half of the effect. But the first layer's sweep never advances: from step 4 000 to the end it reads depth-1 edges at every step. Deep nodes stay at chance. When three-hop nodes finally improve after step 17 500, they do so through query-time reading, whose share at W = 4 rises from 41% to 69%. That route never reaches past three hops. Width then separates the routes further (latent share at W = 8 50% → 23%).

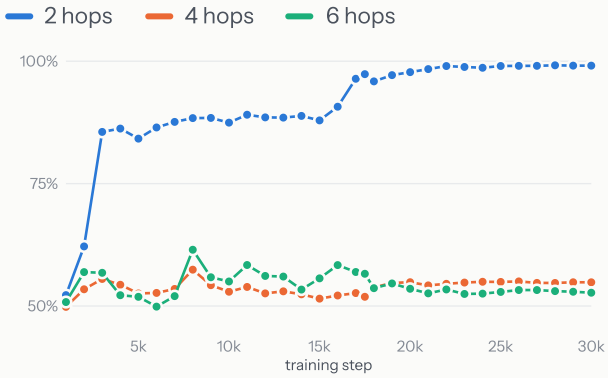
Routes during training, W = 4

Share of the clean counterfactual effect of one branch removal carried by the latent pathway and by query-time graph reading (seed 4, discovery graphs, k = 1).



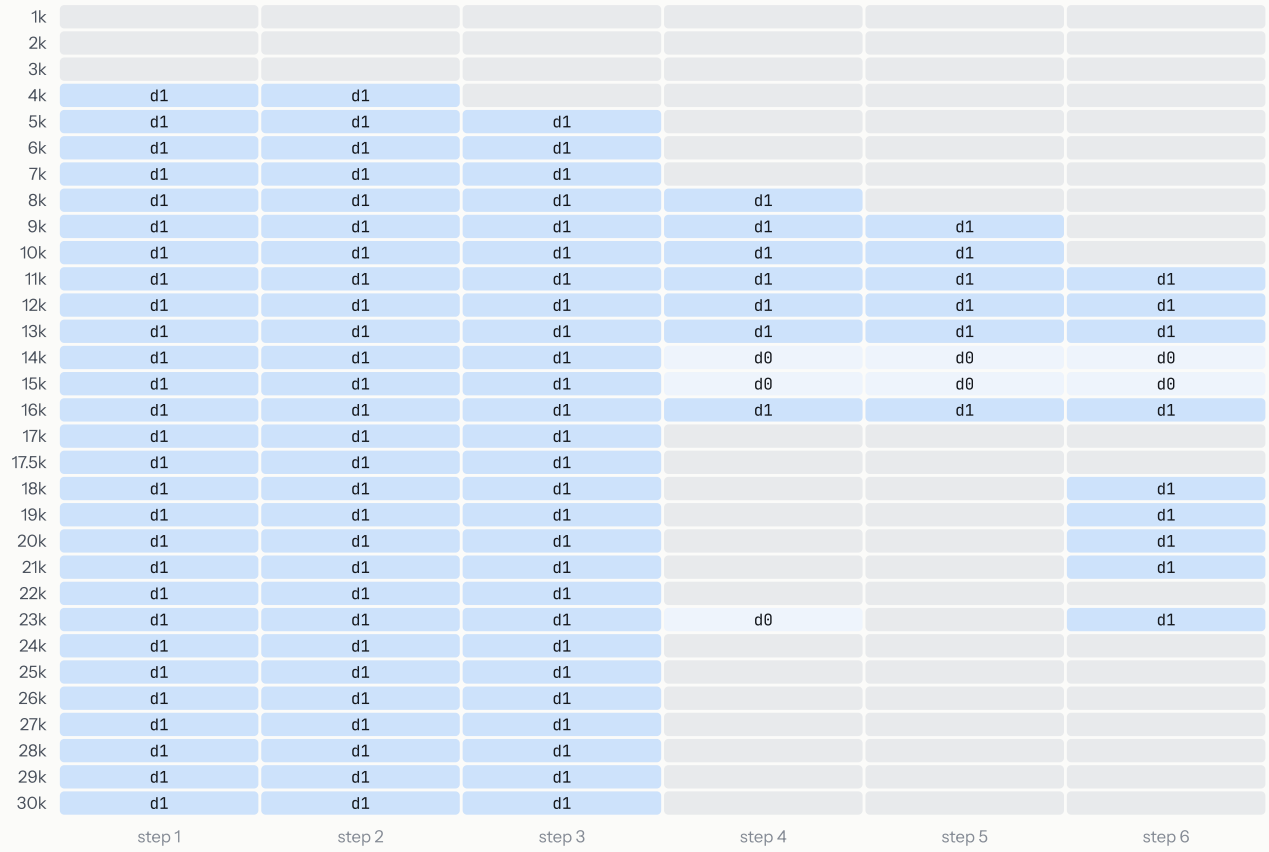
Competence during training

Depth-matched balanced accuracy by hops from start, full budget.



Seed 4: the sweep never advances past depth 1

Depth whose edges layer 1 reads at each latent step (delayed format, W = 4); blank = no depth stands out. Rows: checkpoints.



Hardware incident and an integrity re-run

During the night of 24 September the host crashed with a memory-management bugcheck, and Windows then retired bad RAM regions. Shortly before the crash an analysis job failed on a resampling index of exactly $2^{32} + 11$, a single flipped bit in host memory. One GPU (the RTX 3070) had also logged about 940 context-switch exceptions an hour earlier, which explains a training stall first attributed to memory pressure. The card later passed a bit-exact compute test and a full VRAM pattern test.

To check whether stored results had been silently corrupted, seed 1's fresh-confirmation cache factorial and dose experiments were recomputed from scratch on a different GPU and compared number by number with the stored summary. All 7,034 deterministic quantities agree within 10^{-4}. The only differences (601 numbers) are in the random-subspace null control, whose basis is redrawn from a device RNG.

A second crash followed at about 07:20 (bugcheck 0x133, a DPC watchdog timeout), and at the end of the session the RTX 3070 dropped off the PCI bus ("GPU is lost"), taking the last evaluation job with it; that model had already failed the competence gate. Runs launched after the second crash keep a full training state and resume after a crash.

Training on this rig is not deterministic even with a fixed seed and device, so a seed pins the data stream and initialisation, not the trajectory. About half of all gated training attempts, in both precisions, stalled at the first curriculum stage, with delayed loss on a plateau near 0.565 and upfront

loss at chance. Such runs were stopped at step 4 000 and relaunched under a rule written before any of them was evaluated. Every attempt is kept on disk and listed in the research log.

Claims and the evidence behind them

claim
Recurrent latent computation is needed for the deepest queries
Extra positions or feedback are needed for 3–5 hop queries
Several branches in one saved state can be edited separately
Edits transfer a branch state, not the donor's answers
Probe read-out directions are not the causally used ones
Branch-averaged edits weaken because deep-node counterfactual sensitivity collapses and 2–3-hop nodes move to query-time res
Dilution by magnitude explains the width effect

claim
Branches share a normalised, relative code
Retained cache entries reconstruct an edited branch
Local linear controllability predicts selective-edit outcomes
The mechanism emerges without supervision of intermediate steps
Some recurrent seeds implement an extendable iterative search
The stop is keyed to the step embedding, not the search state
Width limits search fidelity, not speed
The delayed-format route shift does not occur when the target is known
Final-LayerNorm renormalisation sets the per-branch amplitude law

Deviations, limits, and what would change the conclusion

Training scaffold. Plain end-to-end training never left chance (every variant, 3–5k steps). The cause was diagnosed as a saddle: two identity-matching circuits that each need the other before the loss moves. Training therefore used auxiliary PARENT(v) and CHILDREN(v) lookup queries, read against the prefix only and never asked about the start node, annealed to zero by step 15k, plus an accuracy-gated depth curriculum. No loss ever touched latent thoughts or the frontier. The scaffold may favour query-time lookup, and claims about *emergence* of the mechanism should be read with that in mind.

Discovery-driven conditions. The pattern and PCA subspaces were added after discovery showed the probe-filter condition was inert. They were frozen, with the site rule, before confirmation. Two pipeline smoke tests on *intermediate* checkpoints touched confirm-split graphs before sampling was routed through the protocol; no choice was changed as a result, and the event is logged.

Scope. One two-layer architecture, disjoint branches, one intervention interface (the thought fed to the next latent position). The frozen site differs between seeds. Deep-query competence is partial (0.4–0.8 at 5–6 hops), so effects on those nodes act on uncertain baseline predictions. The pause control is a single seed.

What would change the conclusion. (i) A decoder whose weights, used as edit directions, controlled the branch, which would undercut the filter/pattern account. (ii) Selectivity that stayed flat with width at a different interface, such as edits applied to all later positions or to cache entries, which would make the decline an interface artifact rather than interference in the state. (iii) Signed collateral vanishing in more seeds, which would weaken the shared-budget account.

Next. Multi-site edits (thought plus cache) to bound recovery; a formal model of the 1/W attention budget and its predicted selectivity curve; merging branches; a second architecture.

Part II specifics. Graph-route patches change what the model reads as input, so route shares say where the difference is used, not where it is stored. The attention tests bias one layer (all heads) rather than a single identified head. The W = 2 composed-edit recipients have width 0, outside the training range. Insertion at W = 8 creates a width-9 donor. The discovery-derived predictions (P1, P3, P4, P6–P9) confirm discovery observations on fresh graphs rather than testing new hypotheses.

Part III specifics. Each architecture variant is a single training run, so a variant result can reflect that run's trajectory. The held-out seeds address this for the reference recipe only. The length test uses all-chain graphs, whose structure (every branch a chain) essentially never occurs in training; failure to extrapolate could partly reflect that distribution shift rather than the absence of an iterative loop. Stage-1 training failures were handled by a restart rule. This conditions the reported models on competence, as the Part I and II gates already did. Roughly half of all gated training attempts stalled at stage 1, in both precisions. The competence gate is overall balanced accuracy, and one held-out seed that passed it (seed 5) turned out not to use its latent loop at all; a depth-matched gate would have been the better choice.

Reproduce

Everything is in the project directory. `python -m latent.tests` validates the instrument: generator vs exact BFS, single-token counterfactual locality, cache vs full recompute, forked queries vs separate sequences, identity patch exactness. `docs/PROTOCOL.md` and `docs/protocol_frozen.json` hold the pre-registration, `docs/RESEARCH_LOG.md` records every decision including the failed pilots, and `results/summary.json` holds the numbers behind every figure here. Hardware: RTX 4060, RTX 3070, RTX 2070 (8 GB each); about 2 GB and roughly 30–65 minutes per 30k-step run.